WealthWise

Complete Technical Documentation

Smart Mutual Fund Portfolio Management & Financial Goal Planning Platform

Synopsis | SRS v1.0 | System Design (HLD + LLD) | REST API Reference | User Manual

Table of Contents

WealthWise — Complete Technical Documentation

DOCUMENT 01

📘 Project Synopsis

01

📘 WealthWise — Project Synopsis

1.1 Project Title

WealthWise — A Smart Mutual Fund Portfolio Management & Financial Goal Planning Platform

1.2 Problem Statement

India's mutual fund industry manages over ₹61 lakh crore (as of 2025), yet retail investors face critical pain points:

CHALLENGE	IMPACT
Fragmented Portfolio View	Investors hold funds across 5–10 AMCs with no consolidated dashboard
No Real-Time Gain/Loss Tracking	Manual spreadsheets lead to errors in cost-basis and returns calculation
Complex Returns Tracking	FIFO-based cost-basis computation is tedious; investors struggle with accurate gain/loss reporting
Goal Misalignment	No tool links specific holdings to life goals (retirement, education, house)
Data Entry Friction	CAS PDF statements are hard to import; no CSV bulk upload
Lack of Risk Intelligence	No Sharpe ratio, volatility, or drawdown analysis for retail users

Existing platforms (Zerodha Coin, Groww, Kuvera) focus on *buying* funds but provide shallow analytics. **WealthWise bridges this gap** by offering a professional-grade analytics cockpit built for the informed retail investor.

1.3 Objectives

Short-Term Objectives

1. Build a secure, JWT-authenticated platform with email/password registration
2. Enable manual + CSV bulk import of mutual fund transactions

3. Compute real-time holdings with FIFO cost-basis tracking
4. Deliver XIRR-based returns at fund and portfolio level
5. Provide comprehensive portfolio analytics with risk profiling
6. Implement goal-based financial planning with Monte Carlo simulation

Long-Term Objectives

1. CAS PDF auto-import via Apache PDFBox parsing
2. AI-driven spending pattern analysis and fund recommendations
3. SIP step-up optimization engine
4. Multi-currency and NRI portfolio support
5. Mobile-responsive PWA deployment

1.4 Scope of the System

In Scope

- User authentication (signup, login, password reset, change password)
- Mutual fund scheme search (45,000+ AMFI-registered schemes)
- Transaction management (manual entry + CSV import + CAS parsing)
- FIFO lot tracking with automatic cost-basis calculation
- Portfolio dashboard with asset allocation, risk scoring, XIRR
- Financial goal planner with deterministic + Monte Carlo projections
- Goal-fund linking (allocate holdings to specific goals)
- Alert & notification system (SIP due, goal milestones, market alerts)
- User profile management with KYC fields
- Dark/light theme support
- Responsive design for desktop and tablet

Out of Scope (Current Version)

- Direct mutual fund purchase/redemption via BSE STAR or MF Utilities
- Bank account integration for auto-tracking expenses
- Mobile native apps (iOS/Android)
- Multi-tenant admin panel
- Real-time market streaming (WebSocket)

1.5 Key Features Overview

```
┌──────────────────────────────────────────────────────┐
│                  WealthWise Feature Map              │
├──────────────────┬───────────────────────────────────┤
│ Authentication   │ JWT, BCrypt, Password Reset via Email │
│ Fund Discovery   │ AMFI 45k+ scheme search, trigram  │
│ Transaction Engine │ Manual + CSV + CAS import, FIFO lots │
│ Portfolio Analytics │ XIRR, Sharpe, Volatility, Drawdown │
│ Goal Planner     │ Monte Carlo, Deterministic, SIP calc │

│ Alert System     │ SIP due, Goal milestones, Market  │
│ NAV Engine       │ Daily NAV history, auto-fallback  │
│ Risk Profiling   │ 6-level weighted portfolio risk score │
│ SIP Intelligence │ SIP vs Lumpsum analysis per fund  │
└──────────────────┴───────────────────────────────────┘
```

1.6 Technology Stack

Frontend

TECHNOLOGY	VERSION	PURPOSE
React	19.2.4	Component-based UI framework
Vite	8.0.4	Lightning-fast build tool + HMR
React Router DOM	7.14.0	Client-side routing with nested layouts
Recharts	3.8.1	Interactive charting (portfolio, growth, allocation)
Framer Motion	12.38.0	Smooth page transitions and micro-animations
Lucide React	1.8.0	Consistent icon system
Vitest + Playwright	3.1.3 / 1.49.1	Unit + E2E testing

Backend

TECHNOLOGY	VERSION	PURPOSE
Java	17 (LTS)	Runtime platform
Spring Boot	3.2.3	Application framework with auto-configuration
Spring Security	6.x	JWT stateless authentication
Spring Data JPA	3.2.x	ORM with Hibernate for PostgreSQL
Spring Mail	3.2.x	Password reset email delivery
JJWT	0.12.5	JWT token generation and validation
Caffeine Cache	—	In-memory caching for scheme data
Apache PDFBox	3.0.1	CAS PDF statement parsing
Lombok	—	Boilerplate reduction (getters, setters, builders)

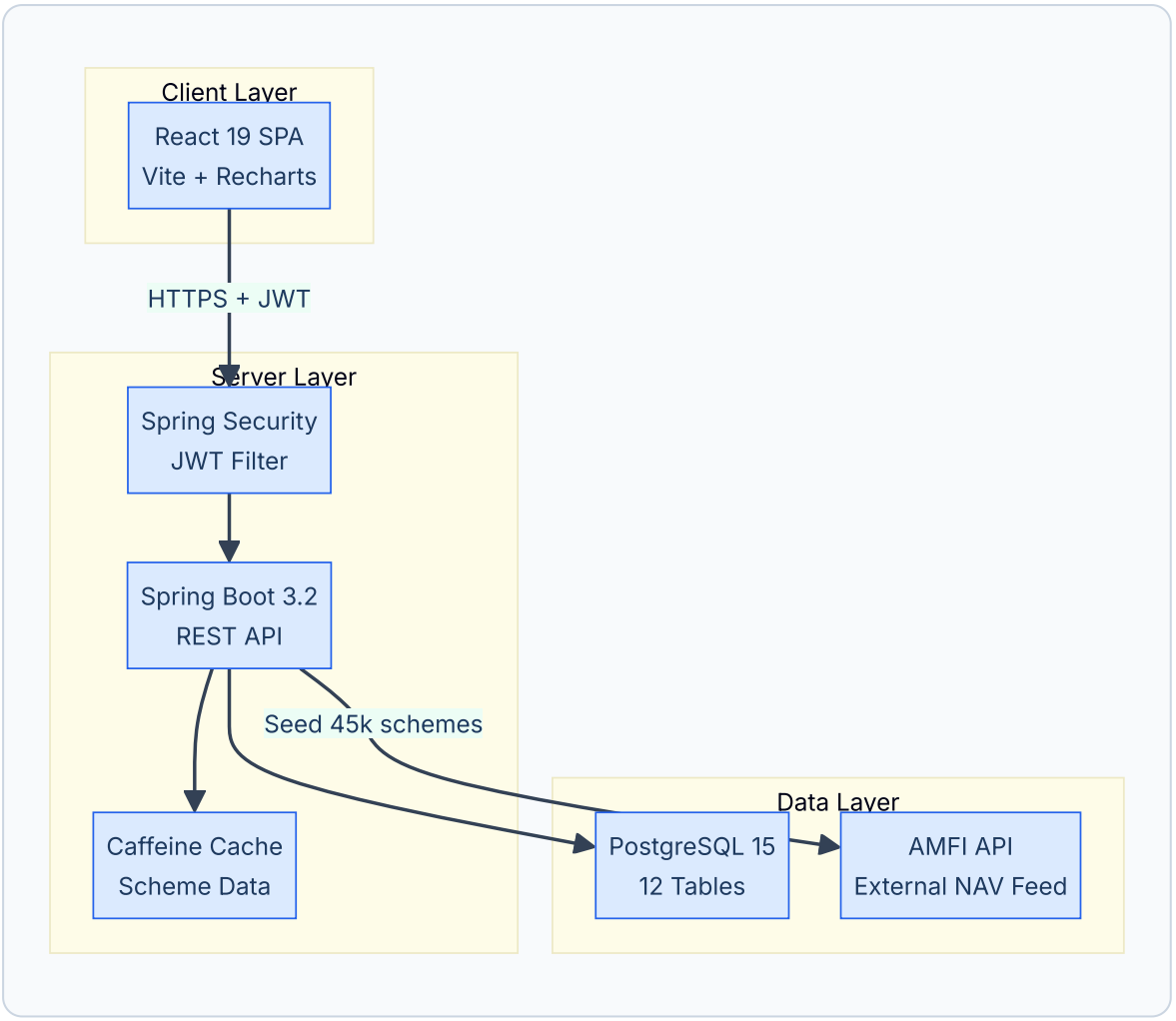
Database

TECHNOLOGY	PURPOSE
PostgreSQL 15+	Primary RDBMS with UUID PKs, JSONB audit log
pgcrypto extension	`gen_random_uuid()` for UUID generation
pg_trgm extension	Trigram-based fuzzy scheme name search
H2 (test scope)	In-memory database for unit tests

DevOps & Deployment

TECHNOLOGY	PURPOSE
Docker	Container image for backend deployment
Microsoft Azure	Production cloud hosting
Render	Secondary deployment target
Git + GitHub	Version control and CI/CD

1.7 Architecture Summary



DOCUMENT 02

📗 Software Requirements Specification

02

📗 WealthWise — Software Requirements Specification (SRS)

2.1 Introduction

2.1.1 Purpose

This Software Requirements Specification defines the complete functional and non-functional requirements for **WealthWise**, a web-based mutual fund portfolio management platform. It serves as the contractual agreement between stakeholders, developers, and QA teams for feature scope, system behavior, and acceptance criteria.

2.1.2 Scope

WealthWise provides retail mutual fund investors with:

- Consolidated portfolio tracking across AMCs
- FIFO-based cost-basis and gain/loss computation
- XIRR and CAGR returns analysis
- Goal-based financial planning with Monte Carlo simulation
- Alert and notification management

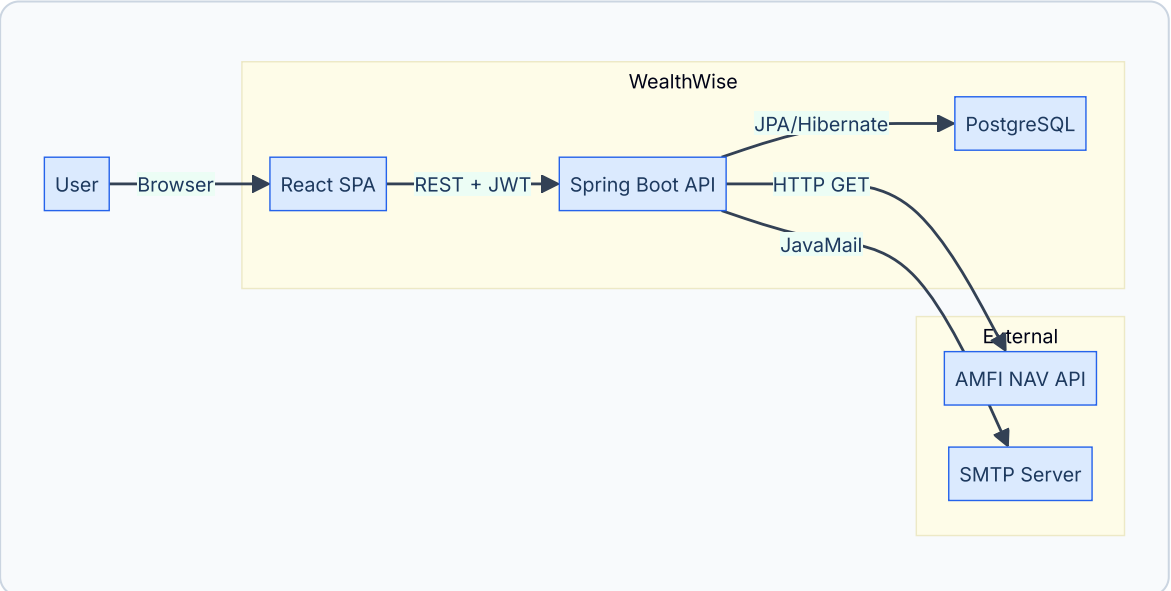
The system does **not** execute actual buy/sell orders. It is an analytics and planning cockpit.

2.1.3 Definitions & Acronyms

TERM	DEFINITION
AMFI	Association of Mutual Funds in India — industry body maintaining scheme registry
NAV	Net Asset Value — per-unit price of a mutual fund scheme
SIP	Systematic Investment Plan — periodic fixed-amount investments
XIRR	Extended Internal Rate of Return — annualized return for irregular cash flows
FIFO	First-In-First-Out — lot consumption order for redemption
STCG	Short-Term Capital Gains — gains on equity funds held < 1 year
LTCG	Long-Term Capital Gains — gains on equity funds held ≥ 1 year
CAS	Consolidated Account Statement — CAMS/KFintech PDF statement
JWT	JSON Web Token — stateless authentication token standard
AMC	Asset Management Company — fund house (e.g., HDFC AMC, SBI AMC)
CAGR	Compound Annual Growth Rate
FY	Financial Year (April–March in India)
KYC	Know Your Customer

2.2 Overall Description

2.2.1 System Overview



WealthWise follows a **three-tier client-server architecture**:

1. **Presentation Tier** — React 19 SPA served via Vite, communicating over HTTPS
2. **Application Tier** — Spring Boot 3.2 REST API with stateless JWT authentication

3. **Data Tier** — PostgreSQL 15 with 12 normalized tables, UUID primary keys, and JSONB audit log

2.2.2 User Classes

USER CLASS	DESCRIPTION	ACCESS LEVEL
Guest	Unauthenticated visitor	Landing page, scheme search, goal engine calculator
Registered User	Authenticated investor	Full dashboard, portfolio, transactions, goals, alerts
System (Automated)	Background scheduler	AMFI scheme seeding, NAV updates, alert generation

2.2.3 Operating Environment

COMPONENT	REQUIREMENT
Client	Modern browser (Chrome 90+, Firefox 88+, Safari 15+, Edge 90+)
Server	Java 17+ runtime, 512MB+ RAM, Linux/Windows/macOS
Database	PostgreSQL 15+ with pgcrypto and pg_trgm extensions
Network	HTTPS (TLS 1.2+), outbound access to AMFI API

2.2.4 Constraints

1. **No Brokerage Integration** — WealthWise does not execute buy/sell orders
2. **Indian Market Only** — Scheme data sourced exclusively from AMFI (Indian mutual funds)
3. **Session** — Stateless JWT; no server-side session store
4. **Concurrent Users** — Designed for up to 500 concurrent users per instance

2.2.5 Assumptions

1. Users have basic understanding of mutual fund terminology
2. NAV data from AMFI is available and updated daily (T+1)
3. Transaction data entered by users is accurate
4. Email service (SMTP) is available for password reset functionality
5. PostgreSQL is pre-provisioned with required extensions

2.3 Functional Requirements

FR-01: Authentication & Authorization

ID	REQUIREMENT	PRIORITY
FR-01.1	System shall allow user registration with email, password (min 8 chars), and optional full name	High
FR-01.2	System shall hash passwords using BCrypt before storage	High
FR-01.3	System shall issue JWT token (HS256) on successful login/signup	High
FR-01.4	System shall validate JWT on every protected endpoint via `JwtAuthFilter`	High
FR-01.5	System shall provide "Forgot Password" flow sending reset link via email	Medium
FR-01.6	System shall allow authenticated users to change password with current password verification	Medium
FR-01.7	System shall return 401 for expired/invalid tokens	High
FR-01.8	Public endpoints (`/api/auth/` , **`/api/schemes/search`** , **`/api/nav/`** , `/api/learn/**`) shall be accessible without authentication	High

FR-02: Scheme Discovery (AMFI Integration)

ID	REQUIREMENT	PRIORITY
FR-02.1	System shall maintain a `scheme_master` table with 45,000+ AMFI-registered schemes	High
FR-02.2	System shall provide trigram-based fuzzy search on scheme names (`GET /api/schemes/search?q=...`)	High
FR-02.3	System shall auto-seed schemes from AMFI API on application startup	Medium
FR-02.4	System shall store scheme metadata: AMC name, category, sub-category, plan type, risk level, last NAV	High
FR-02.5	System shall provide unique AMC listing for filter dropdowns	Low

FR-03: Transaction Management

ID	REQUIREMENT	PRIORITY
FR-03.1	System shall support manual transaction entry with scheme code, type, date, amount, NAV, units	High
FR-03.2	System shall support 11 transaction types: PURCHASE_LUMPSUM, PURCHASE_SIP, REDEMPTION, SWITCH_IN, SWITCH_OUT, STP_IN, STP_OUT, SWP, DIVIDEND_PAYOUT, DIVIDEND_REINVEST, REVERSAL	High
FR-03.3	System shall auto-calculate units when NAV and amount are provided (`units = amount / NAV`)	High
FR-03.4	System shall auto-lookup NAV from `nav_daily` table when user omits it	Medium
FR-03.5	System shall support CSV bulk import with intelligent column mapping (supports 6+ date formats, flexible column names)	High
FR-03.6	CSV import shall return per-row status (OK/FAILED) with error messages	Medium
FR-03.7	System shall auto-generate deterministic folio numbers for manual transactions	Medium
FR-03.8	Transactions shall be immutable once created (no edit, only new entries)	High

FR-04: FIFO Lot Tracking

ID	REQUIREMENT	PRIORITY
FR-04.1	On BUY-type transactions, system shall create an `investment_lot` recording purchase date, units, cost NAV	High
FR-04.2	On SELL-type transactions, system shall apply FIFO order: consume oldest lots first	High
FR-04.3	System shall track `units_remaining` per lot and mark `is_fully_redeemed` when exhausted	High
FR-04.4	System shall compute holding period (days) per lot for STCG/LTCG classification (< 365 days = STCG)	High
FR-04.5	System shall record per-transaction STCG gain, LTCG gain, STCG units, LTCG units	High

FR-05: Portfolio & Holdings

ID	REQUIREMENT	PRIORITY
FR-05.1	System shall compute current holdings by aggregating active (non-redeemed) lots per scheme	High
FR-05.2	System shall display: total units, average cost NAV, invested value, current value, gain/loss, gain%	High
FR-05.3	System shall compute XIRR per fund using lot-level cash flows	High
FR-05.4	Portfolio summary shall include: total invested, current value, total gain, portfolio XIRR, risk score	High
FR-05.5	System shall compute asset allocation breakdown (Equity, Debt, Hybrid, Commodity, Other)	High
FR-05.6	System shall compute portfolio risk score (1-6 weighted average) with human-readable label	Medium
FR-05.7	System shall provide monthly portfolio growth timeline	Medium
FR-05.8	System shall provide risk profile analytics: Sharpe ratio, volatility, max drawdown, diversification score	Medium
FR-05.9	System shall provide SIP vs Lumpsum intelligence analysis per fund	Low

FR-06: Financial Goals

ID	REQUIREMENT	PRIORITY
FR-06.1	System shall support CRUD operations on financial goals	High
FR-06.2	Goals shall have: name, type, priority (High/Medium/Low), target amount, target date, inflation rate, expected return	High
FR-06.3	System shall run deterministic projection (compound growth) for each goal	High
FR-06.4	System shall run Monte Carlo simulation (5000 iterations) for probability-of-success	High
FR-06.5	System shall compute required SIP to reach goal target	Medium
FR-06.6	System shall allow linking specific fund holdings to goals with allocation percentage	Medium
FR-06.7	Goal corpus shall auto-refresh from linked holdings' current value on every GET	Medium
FR-06.8	Goals shall support status transitions: Active → Achieved / Abandoned	Low

FR-07: Alerts & Notifications

ID	REQUIREMENT	PRIORITY
FR-07.1	System shall store alerts with title, description, severity (Info/Watch/Action), and read status	High
FR-07.2	System shall provide unread alert count for badge display	High
FR-07.3	System shall support mark-as-read and delete operations per alert	Medium
FR-07.4	Notification preferences shall be configurable: SIP due, goal milestones, daily digest, market alerts (in-app / email toggle)	Medium

FR-08: User Profile

ID	REQUIREMENT	PRIORITY
FR-09.1	System shall store extended profile: full name, phone, PAN, DOB, risk tolerance, nominee info	Medium
FR-09.2	System shall support avatar URL and currency format preference	Low

2.4 Non-Functional Requirements

NFR-01: Performance

METRIC	REQUIREMENT
API Response Time	≤ 500ms for 95th percentile on standard queries
Portfolio Calculation	≤ 2 seconds for user with 50 holdings
Monte Carlo Simulation	≤ 1 second for 5,000 iterations
Scheme Search	≤ 200ms with trigram index
CSV Import	≤ 5 seconds for 500-row file

NFR-02: Scalability

- Horizontal scaling via containerized deployment (Docker)
- Caffeine in-memory cache for frequently accessed scheme data
- Database indexing on all foreign keys and common query patterns
- Stateless JWT enables load-balancer distribution

NFR-03: Security

CONTROL	IMPLEMENTATION
Password Hashing	BCrypt with default strength (10 rounds)
Token Auth	JWT HS256, configurable expiry
CORS	Configurable allowed origins
CSRF	Disabled (stateless API)
Input Validation	Spring Validation annotations
SQL Injection	JPA parameterized queries
Authorization	Per-request userId ownership checks
Audit Trail	Immutable `audit_log` table with JSONB diff

NFR-04: Availability

- Target uptime: 99.5%
- Health check endpoint: `GET /api/auth/health`
- Graceful degradation: NAV lookup falls back to `scheme_master.last_nav` when `nav_daily` is empty

NFR-05: Usability

- Dark/Light theme toggle with system preference detection
- Responsive layout supporting 1024px+ viewports
- Animated page transitions via Framer Motion
- Consistent icon language via Lucide React

NFR-06: Reliability

- Transactions are immutable — no destructive edits
- FIFO lot state is persisted atomically with transactions
- Foreign keys with `ON DELETE CASCADE` ensure referential integrity
- Migration-safe schema using `IF NOT EXISTS` and `ALTER TABLE ADD COLUMN IF NOT EXISTS`

2.5 Business Rules

RULE ID	DESCRIPTION
BR-01	A user can only view/modify their own data (enforced by `userId` filter on every query)
BR-02	Transactions are immutable once created — corrections require new reversal transactions
BR-03	FIFO lot consumption is mandatory on SELL-type transactions
BR-04	Goal priority must be one of: High, Medium, Low
BR-05	Goal status transitions: Active → Achieved, Active → Abandoned
BR-06	Alert severity levels: Info (informational), Watch (attention needed), Action (immediate action required)
BR-09	Password minimum length: 8 characters
BR-10	Folio number auto-generation format: `WW` + 6-char userId hash + 6-digit scheme code

2.6 Use Cases

UC-01: User Registration

FIELD	DETAIL
Use Case ID	UC-01
Name	User Registration
Actor	Guest
Preconditions	User has a valid email address; user is not already registered
Trigger	User clicks "Sign Up" on landing page

Main Flow:

1. User navigates to `/signup`
2. User enters email, password (min 8 chars), and optional full name
3. System validates email format and password length
4. System checks email uniqueness in `users` table
5. System hashes password with BCrypt
6. System creates `AppUser` record with UUID
7. System generates JWT token
8. System returns `{ token, user: { id, email, fullName } }`
9. Frontend stores token in localStorage and redirects to `/dashboard`

Alternate Flows:

- **A1:** Email already registered → Return 400 `"Email already in use"`

- **A2:** Password too short → Return 400 `"Password must be at least 8 characters"`

Postconditions: User account exists; JWT is valid; user is redirected to dashboard.

UC-02: User Login

FIELD	DETAIL
Use Case ID	UC-02
Name	User Login
Actor	Guest
Preconditions	User has an existing account
Trigger	User clicks "Log In"

Main Flow:

1. User enters email and password on `/login`
2. System looks up user by email
3. System verifies password hash with BCrypt
4. System generates JWT token
5. System returns `{ token, user }` with 200 OK
6. Frontend stores token and navigates to `/dashboard`

Alternate Flows:

- **A1:** Email not found → Return 401 `"Invalid credentials"`
- **A2:** Wrong password → Return 401 `"Invalid credentials"`

UC-03: Record Investment Transaction

FIELD	DETAIL
Use Case ID	UC-03
Name	Record Investment Transaction
Actor	Registered User
Preconditions	User is authenticated; scheme exists in scheme_master

Main Flow:

1. User navigates to "New Investment" page
2. User searches for fund using autocomplete (trigram search)
3. User selects transaction type (SIP, Lumpsum, Redemption, etc.)
4. User enters date, amount (and optionally NAV, units)

5. System auto-fetches NAV from `nav_daily` if not provided
6. System auto-calculates units = amount / NAV
7. System creates `InvestmentTransaction` record
8. For BUY: System creates `InvestmentLot` with purchase date and cost basis
9. For SELL: System applies FIFO, consuming oldest lots, computing STCG/LTCG gains
10. System returns saved transaction

Alternate Flows:

- **A1:** Missing required fields → Return 400 with field-specific message
- **A2:** Invalid date format → Return 400 `"Invalid date. Use yyyy-MM-dd"`
- **A3:** NAV not available → Use `scheme_master.last_nav` as fallback

UC-04: View Portfolio Summary

FIELD	DETAIL
Use Case ID	UC-04
Name	View Portfolio Summary
Actor	Registered User
Preconditions	User has at least one transaction recorded

Main Flow:

1. User navigates to `/dashboard/portfolio`
2. Frontend calls `GET /api/holdings/summary`
3. System aggregates all active lots by scheme code
4. System computes per-fund: invested value, current value, gain/loss, XIRR
5. System computes portfolio: total invested, total current, gain%, portfolio XIRR, risk score
6. System computes asset allocation by broad category
7. Frontend renders summary cards, allocation pie chart, holdings table

---

UC-05: Create Financial Goal

FIELD	DETAIL
Use Case ID	UC-05
Name	Create Financial Goal
Actor	Registered User
Preconditions	User is authenticated

Main Flow:

1. User navigates to "New Goal" page
2. User selects goal type (Retirement, Education, House, Emergency, Custom)
3. User enters target amount, target date, inflation rate, expected return
4. System creates `FinancialGoal` record
5. System runs deterministic projection and Monte Carlo simulation
6. System returns goal with projected values and success probability

---

UC-06: Import Transactions via CSV

FIELD	DETAIL
Use Case ID	UC-06
Name	Bulk Import via CSV
Actor	Registered User
Preconditions	User has a CSV file with transaction data

Main Flow:

1. User navigates to "Import" page and uploads CSV file
2. System reads CSV with BOM handling
3. System maps columns flexibly (supports aliases: `scheme_code`, `amfi_code`, `schemecode`, etc.)
4. System parses dates in 6 formats (yyyy-MM-dd, dd-MM-yyyy, dd/MM/yyyy, etc.)
5. System normalizes transaction types (SIP → PURCHASE_SIP, BUY → PURCHASE_LUMPSUM, etc.)
6. For each row: creates transaction, applies FIFO lot logic
7. System returns summary: `{ imported, failed, skipped, rows: [...] }`

Alternate Flows:

- **A1:** Non-CSV file → Return 400 `"Only .csv files are supported"`
- **A2:** Missing scheme_code in row → Row marked FAILED with message
- **A3:** Unrecognized date format → Row marked FAILED

DOCUMENT 03

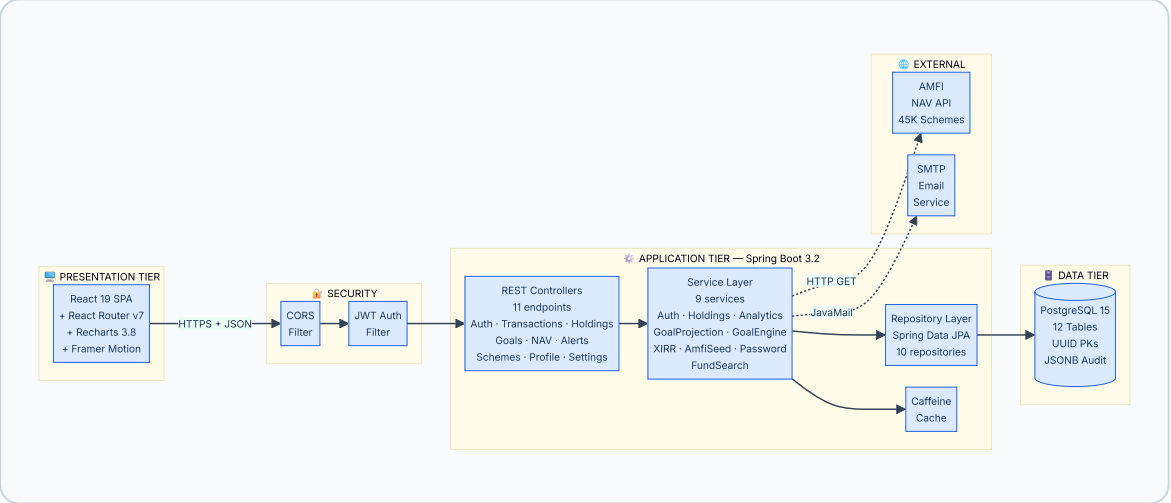
System Design (HLD + LLD)

03

WealthWise — System Design (HLD + LLD)

3.1 Architecture Diagram

WealthWise employs a **monolithic client-server architecture** with clear separation of concerns across three tiers. The backend follows a layered pattern (Controller → Service → Repository) and the frontend uses a component-based SPA architecture.

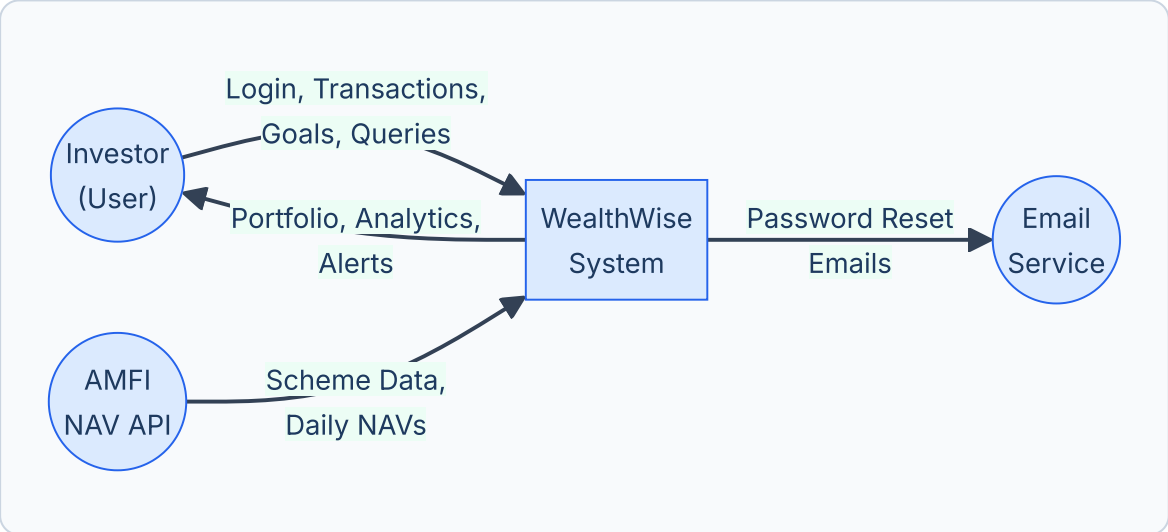


Layered Architecture Summary

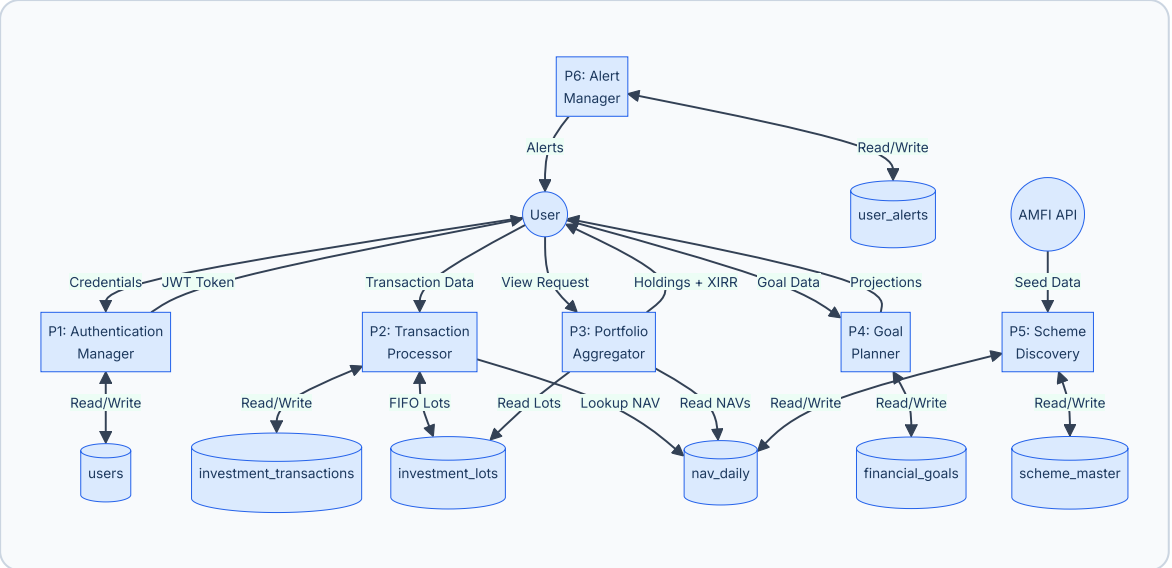
LAYER	TECHNOLOGY	COMPONENTS
Presentation	React 19, Vite 8, Recharts, Framer Motion	19 pages, 6 components, 5 lib modules
Security	Spring Security, JJWT 0.12.5, BCrypt	CORS filter, JWT auth filter, stateless sessions
Controllers	Spring MVC REST	12 controllers exposing 35+ endpoints
Services	Spring Service beans	9 services (Auth, Holdings, Analytics, XIRR, GoalEngine, AmfiSeed, etc.)
Repositories	Spring Data JPA, Hibernate	10 repositories with custom JPQL queries
Database	PostgreSQL 15	12 tables, pgcrypto UUIDs, pg_trgm fuzzy search, JSONB audit
External	AMFI API, SMTP	Scheme registry (45K+ funds), password reset emails

3.2 Data Flow Diagrams

DFD Level 0 — Context Diagram

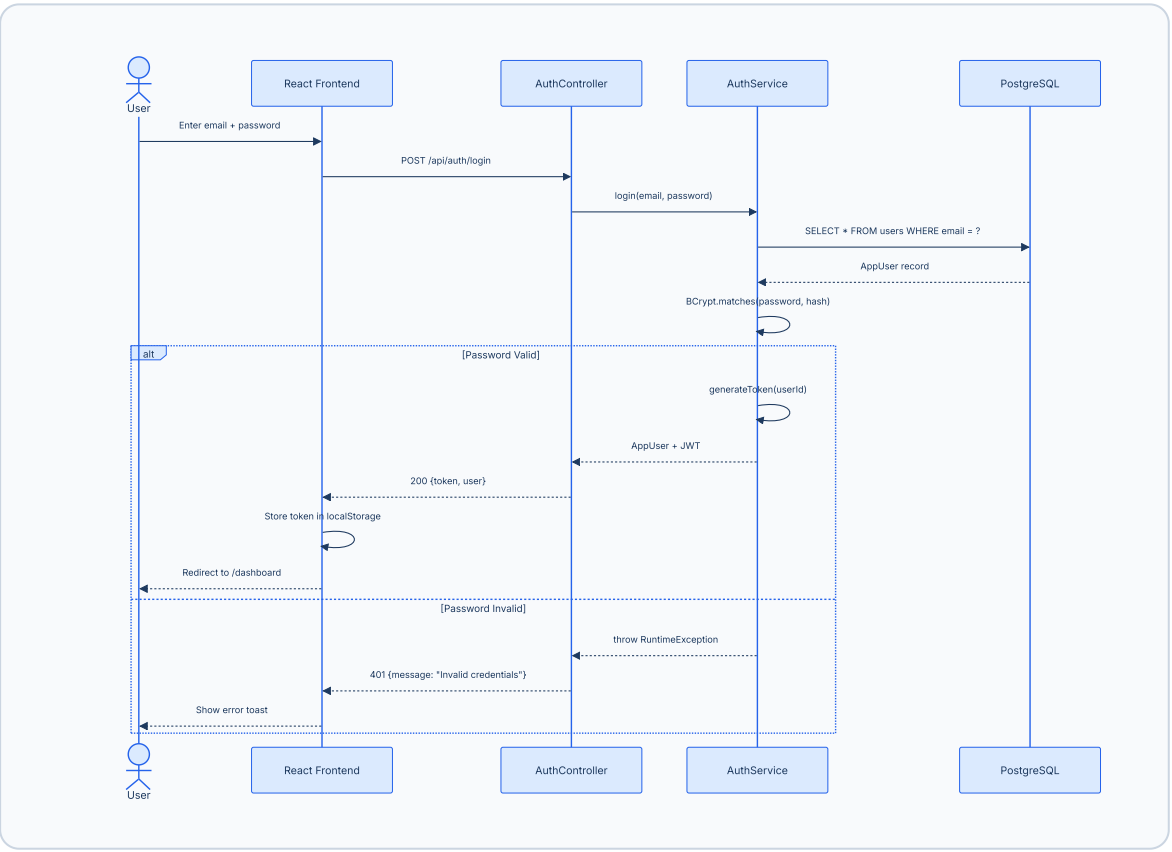


DFD Level 1 — Major Processes

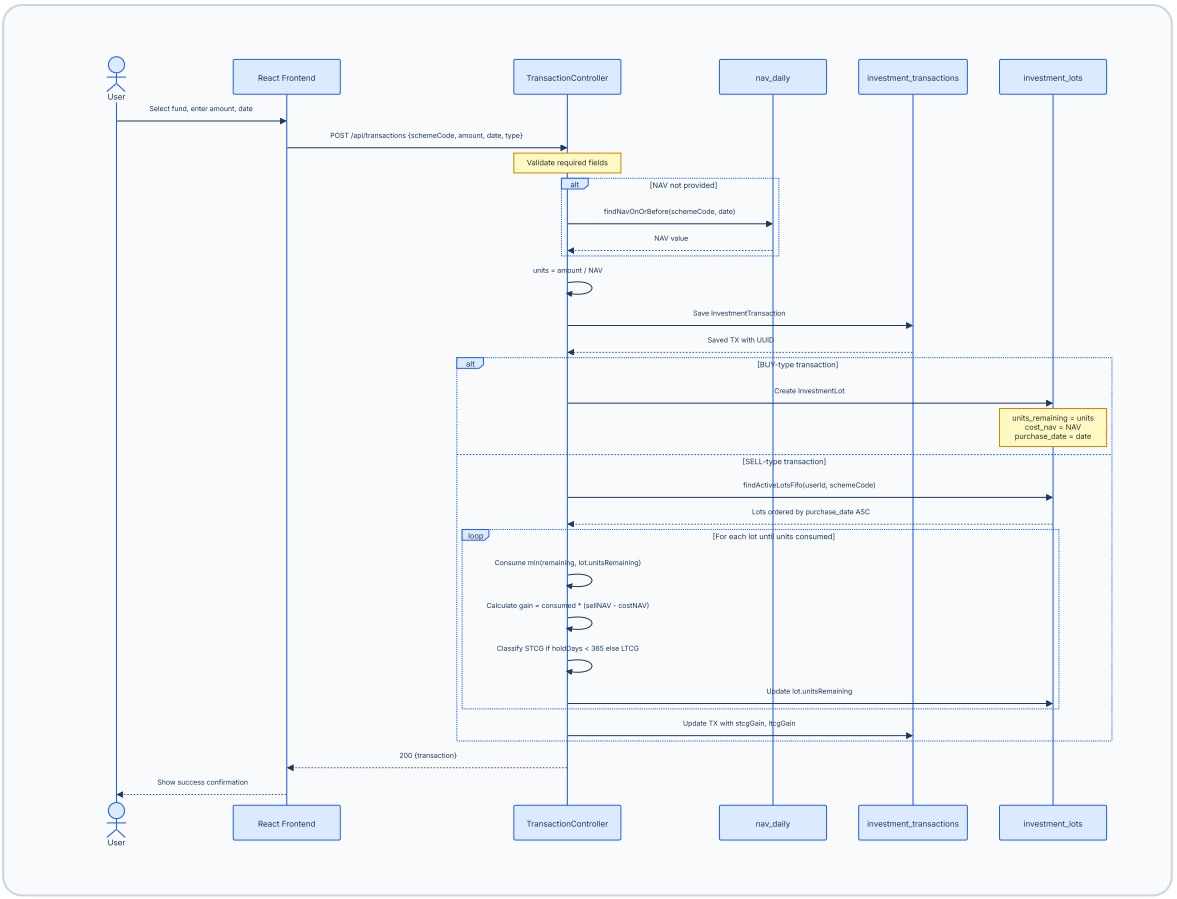


3.3 Sequence Diagrams

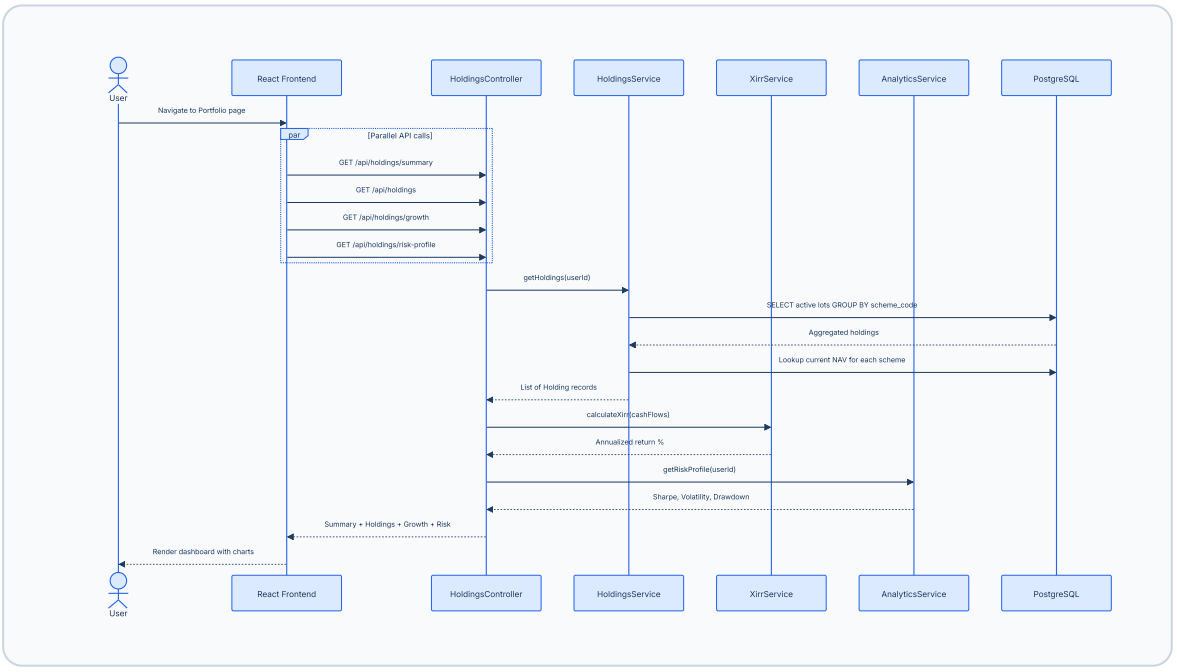
3.3.1 Login Flow



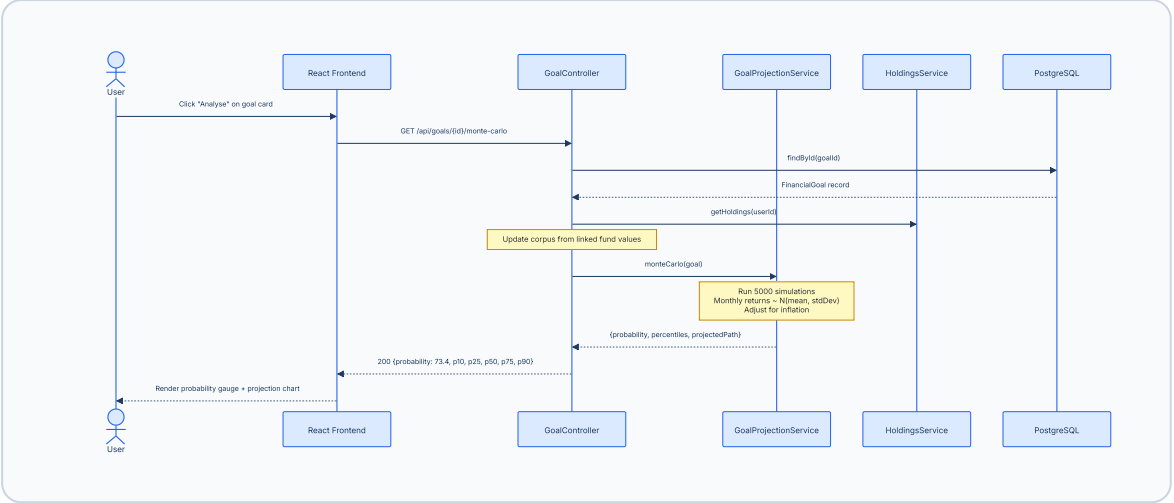
3.3.2 Transaction Entry + FIFO Lot Logic



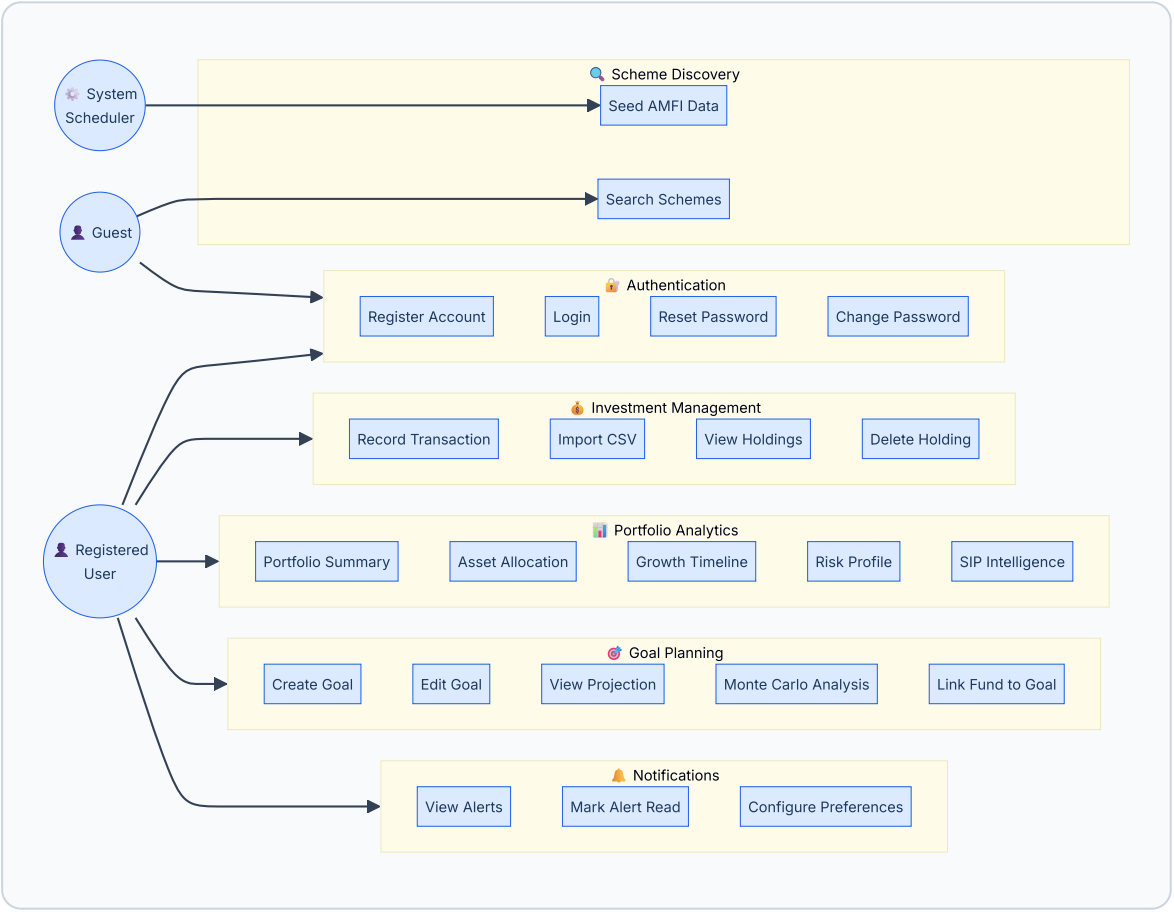
3.3.3 Portfolio Summary & Analytics



3.3.4 Goal Projection (Monte Carlo)



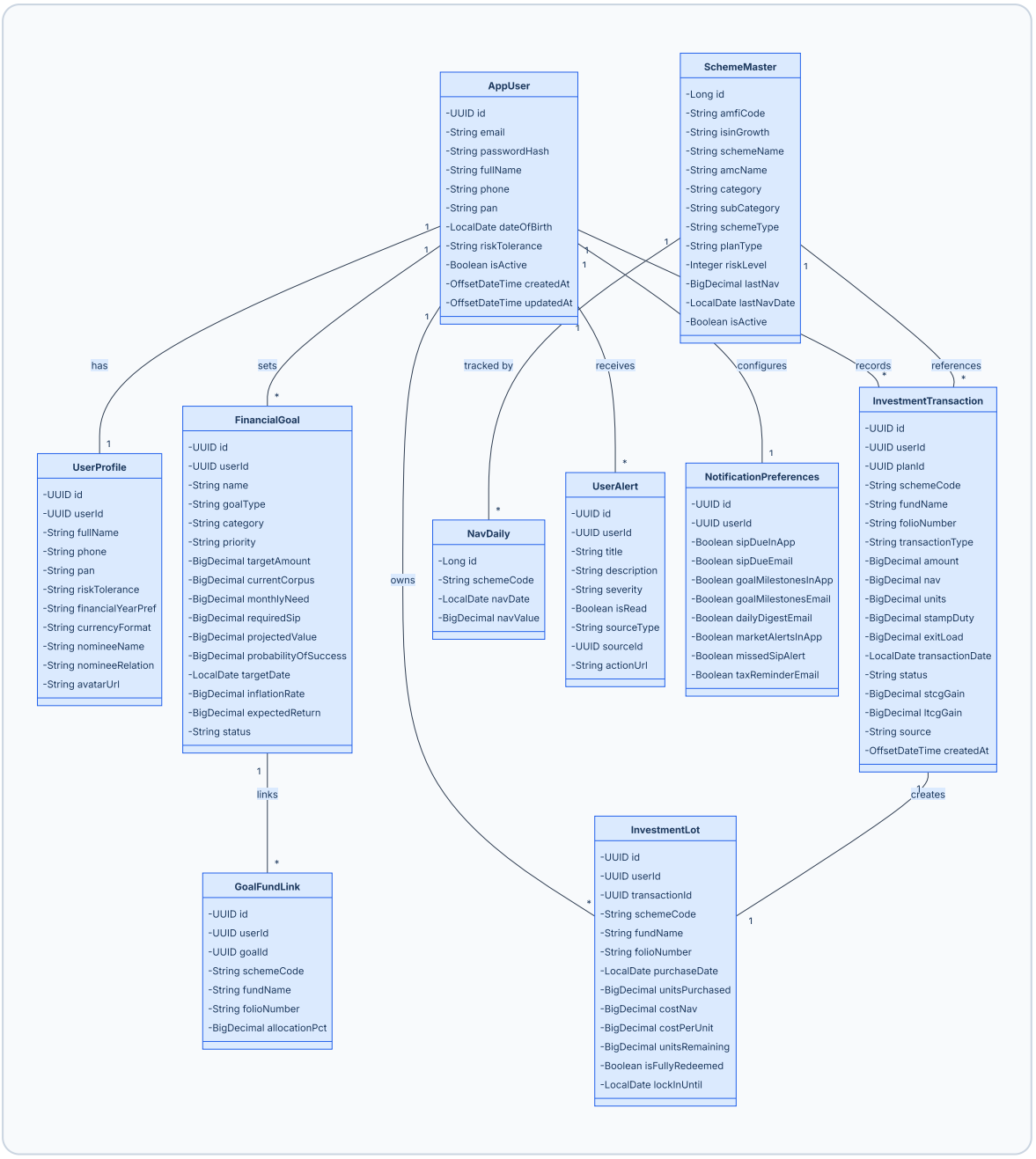
3.4 Use Case Diagram



Use Case Summary by Actor

ACTOR	MODULE	USE CASES
Guest	Authentication	Register, Login, Reset Password
Guest	Scheme Discovery	Search 45K+ AMFI schemes
Registered User	Authentication	Change Password
Registered User	Investment Management	Record Transaction, Import CSV, View Holdings, Delete Holding
Registered User	Portfolio Analytics	Summary, Allocation, Growth, Risk, SIP Intelligence
Registered User	Goal Planning	Create, Edit, Projection, Monte Carlo, Link Fund
Registered User	Notifications	View Alerts, Mark Read, Configure Preferences
System Scheduler	Scheme Discovery	Seed AMFI Data (auto on startup)

3.5 Class Diagram



3.6 Module Breakdown

Frontend Modules

MODULE	PATH	RESPONSIBILITY
Auth Shell	`components/AuthShell.jsx`	Login/Signup page layout with branding
Dashboard Shell	`components/DashboardShell.jsx`	Sidebar navigation, topbar, theme toggle
Landing Page	`pages/LandingPage.jsx`	Marketing landing with feature showcase
Dashboard Page	`pages/DashboardPage.jsx`	Portfolio summary cards, charts, recent activity
Portfolio Page	`pages/PortfolioPage.jsx`	Holdings table, allocation, risk, growth charts
Transactions Page	`pages/TransactionsPage.jsx`	Transaction history with filters and pagination
Goals Page	`pages/GoalsPage.jsx`	Goal cards with progress, analysis modal
Goal Wizard	`components/GoalWizard.jsx`	Multi-step goal creation form
Goal Analysis Modal	`components/GoalAnalysisModal.jsx`	Monte Carlo + projection visualization
Notifications Page	`pages/NotificationsPage.jsx`	Alert list with read/dismiss actions
Auth Context	`lib/auth.jsx`	React context for JWT token + user state
API Client	`lib/api.js`	Centralized fetch wrapper with auth headers
Theme Provider	`lib/theme.jsx`	Dark/light mode context and CSS variable injection
Goal Helpers	`lib/goalHelpers.js`	Client-side goal calculation utilities
MF API	`lib/mfApi.js`	AMFI external API integration helpers

Backend Services

SERVICE	CLASS	RESPONSIBILITY
AuthService	`AuthService.java`	User registration, login, password management, JWT generation
HoldingsService	`HoldingsService.java`	Aggregate lots into holdings with current NAV valuation
AnalyticsService	`AnalyticsService.java`	Growth timeline, risk profile (Sharpe, volatility, drawdown), SIP intelligence
GoalProjectionService	`GoalProjectionService.java`	Deterministic compound projection for goals
GoalEngineService	`GoalEngineService.java`	Monte Carlo simulation (5000 runs), required SIP calculator
XirrService	`XirrService.java`	Newton-Raphson XIRR solver for irregular cash flows
AmfiSeedService	`AmfiSeedService.java`	Fetch + parse AMFI text file, upsert 45k+ schemes + NAVs
PasswordResetService	`PasswordResetService.java`	Token generation, email dispatch, token validation
FundSearchService	`FundSearchService.java`	Trigram-based scheme name search

Database Layer (12 Tables)

TABLE	RECORDS	PURPOSE
`users`	User accounts	Authentication source of truth
`user_profiles`	Extended KYC	Profile, PAN, nominee, preferences
`investment_plans`	SIP/Lumpsum plans	Planned investment tracker
`investment_transactions`	Immutable ledger	All buy/sell/switch records
`investment_lots`	FIFO cost basis	Per-lot unit tracking
`financial_goals`	Life goals	Target amounts, dates, projections
`goal_fund_links`	Goal ↔ Fund map	Allocation percentage per scheme
`scheme_master`	45,000+ schemes	AMFI registry with metadata
`nav_daily`	Historical NAVs	Daily NAV price store
`user_alerts`	Notifications	Severity-based alert queue
`notification_preferences`	Per-user toggles	In-app + email notification config
`audit_log`	Immutable trail	JSONB mutation audit

3.7 Technology Choices Justification

TECHNOLOGY	RATIONALE
React 19	Latest stable with concurrent features; largest ecosystem; component reusability
Vite 8	10x faster HMR than Webpack; native ESM; minimal config
Spring Boot 3.2	Production-grade Java framework; auto-configuration; massive enterprise adoption
PostgreSQL 15	ACID compliance; UUID native support; JSONB for audit log; pg_trgm for fuzzy search
JWT (JJWT 0.12)	Stateless auth eliminates session store; scales horizontally; standard RFC 7519
BCrypt	Industry-standard adaptive hashing; resistant to rainbow table + brute force
Caffeine Cache	Fastest JVM cache; near-zero latency for scheme lookups
Recharts	React-native charting; declarative API; responsive; lightweight
Framer Motion	Production-grade React animation library; layout animations; gesture support
Docker	Consistent deployment across dev/staging/prod; Azure/Render compatible
Lombok	60% boilerplate reduction; cleaner entity classes

3.8 Core Function Logic

3.8.1 FIFO Lot Consumption (Pseudocode)

```
FUNCTION applyFifo(userId, schemeCode, unitsToRedeem, redeemDate, sellNAV):
    remaining = unitsToRedeem
    stcgGain = 0, ltcgGain = 0

    lots = DB.findActiveLots(userId, schemeCode, ORDER BY purchaseDate ASC)

    FOR EACH lot IN lots:
        IF remaining ⩽ 0: BREAK

        consume = MIN(remaining, lot.unitsRemaining)
        gainPerUnit = sellNAV - lot.costPerUnit
        totalGain = consume * gainPerUnit
        holdDays = DAYS_BETWEEN(lot.purchaseDate, redeemDate)

        IF holdDays ⩾ 365:
            ltcgGain += totalGain    // Long-term
        ELSE:
            stcgGain += totalGain    // Short-term

        lot.unitsRemaining -= consume
        IF lot.unitsRemaining ⩽ 0:
            lot.isFullyRedeemed = TRUE
        DB.save(lot)
        remaining -= consume

    RETURN { stcgGain, ltcgGain }
```

3.8.2 XIRR Calculation (Newton-Raphson)

```
FUNCTION calculateXirr(cashFlows[]):
    // cashFlows = [{date, amount}, ...] where outflows are negative
    guess = 0.1  // Initial 10% guess

    FOR i = 1 TO 100:  // Max iterations
        f = 0, fPrime = 0
        FOR EACH cf IN cashFlows:
            years = DAYS_BETWEEN(cashFlows[0].date, cf.date) / 365.25
            denominator = (1 + guess) ^ years
            f += cf.amount / denominator
            fPrime -= years * cf.amount / ((1 + guess) ^ (years + 1))

        newGuess = guess - f / fPrime
        IF ABS(newGuess - guess) < 1e-7: RETURN newGuess
        guess = newGuess

    RETURN guess  // Best approximation
```

3.8.3 Monte Carlo Goal Simulation

```
FUNCTION monteCarlo(goal, simulations=5000):
    results = []
    monthsToGoal = MONTHS_BETWEEN(NOW, goal.targetDate)
    monthlyReturn = goal.expectedReturn / 12 / 100
    monthlyStdDev = monthlyReturn * 0.5  // Assumed volatility
    monthlyInflation = goal.inflationRate / 12 / 100

    FOR sim = 1 TO simulations:
        portfolio = goal.currentCorpus
        FOR month = 1 TO monthsToGoal:
            randomReturn = NORMAL_RANDOM(monthlyReturn, monthlyStdDev)
            portfolio = portfolio * (1 + randomReturn)
            portfolio += goal.monthlyNeed  // Monthly SIP

        // Adjust target for inflation
        inflatedTarget = goal.targetAmount * (1 + monthlyInflation) ^ monthsToGoal
        results.APPEND(portfolio ≥ inflatedTarget)

    probability = COUNT(results WHERE TRUE) / simulations * 100
    RETURN { probability, percentiles: P10, P25, P50, P75, P90 }
```

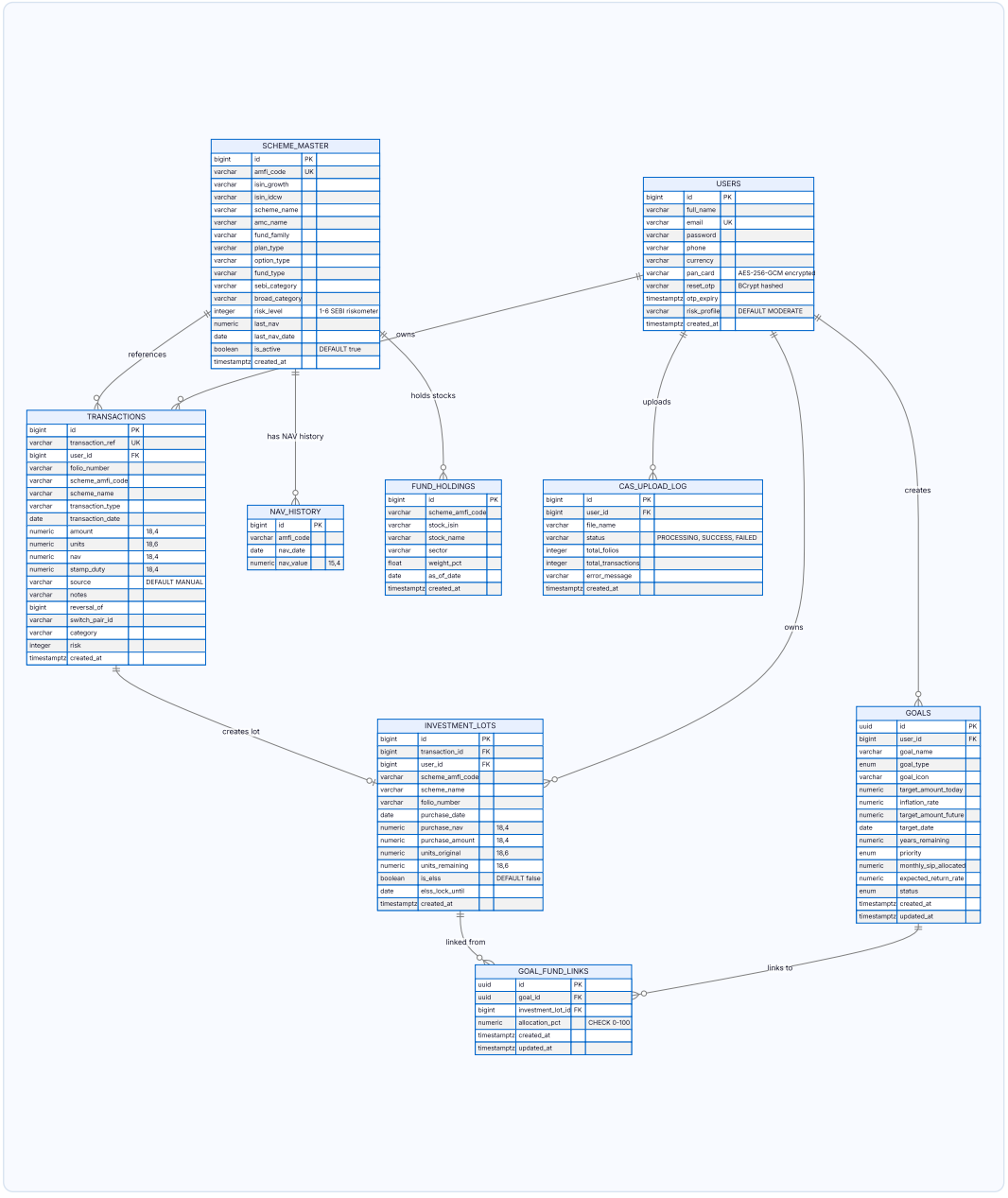
3.9 Data Flow Explanation

End-to-End: From Transaction to Portfolio Dashboard

Database Schema Design

9. Database Schema Design

9.1 Entity-Relationship Diagram



9.2 Table Details and Constraints

TABLE	ROWS (EST.)	PRIMARY KEY	UNIQUE CONSTRAINTS	FOREIGN KEYS	KEY INDEXES
`users`	~10K	`id` (BIGSERIAL)	`email`	—	email
`transactions`	~500K	`id` (BIGSERIAL)	`transaction_ref`	`user_id` → users	(user_id), (user_id, scheme_amfi_code), (user_id, folio_number), (transaction_date)
`investment_lots`	~500K	`id` (BIGSERIAL)	—	`transaction_id` → transactions	(user_id), (user_id, scheme_amfi_code), (user_id, folio_number), (purchase_date)
`scheme_master`	~50K	`id` (BIGSERIAL)	`amfi_code`	—	(amfi_code), (scheme_name), (amc_name), (broad_category)
`nav_history`	~10M	`id` (BIGSERIAL)	`(amfi_code, nav_date)`	—	(amfi_code, nav_date)
`fund_holdings`	~100K	`id` (BIGSERIAL)	—	—	(scheme_amfi_code), (stock_name)
`goals`	~50K	`id` (UUID)	—	`user_id` → users	(user_id)
`goal_fund_links`	~100K	`id` (UUID)	—	`goal_id` → goals, `investment_lot_id` → investment_lots	(goal_id), (investment_lot_id)
`cas_upload_log`	~20K	`id` (BIGSERIAL)	—	`user_id` → users	(user_id)

9.3 Precision Strategy

DATA TYPE	PRECISION	USAGE
`NUMERIC(18,4)`	18 total digits, 4 decimal places	Monetary amounts (INR), NAV values, stamp duty
`NUMERIC(18,6)`	18 total digits, 6 decimal places	Mutual fund units (MFs typically report to 3–4 decimals; 6 provides headroom)
`NUMERIC(15,4)`	15 total digits, 4 decimal places	Historical NAV values (max NAV ever ≈ ₹9,999,999.9999)

DOCUMENT 04

REST API Documentation

04

WealthWise — API Documentation

Base URL: `https://api.wealthwise.app` (Production) | `http://localhost:8080` (Development)
Authentication: Bearer JWT Token (obtained via `/api/auth/login` or `/api/auth/signup`)
Content-Type: `application/json` (unless specified otherwise) **Document Version:** 1.0 | April 2026

Authentication Headers

All protected endpoints require:

```
Authorization: Bearer <jwt_token>
```

Public endpoints (no token required): `/api/auth/`, **`/api/schemes/search`**, **`/api/schemes/stats`**, **`/api/schemes/amcs`**, **`/api/nav/`**, `/api/learn/**`

---

1. Auth APIs

1.1 POST `/api/auth/signup`

Description: Register a new user account.

Request Body:

```
{
  "email": "investor@example.com",
  "password": "SecureP@ss123",
  "fullName": "Bhuvan Nagesh"
}
```

Success Response (200):

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "user": {
    "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "email": "investor@example.com",
    "fullName": "Bhuvan Nagesh"
  }
}
```

Error Responses:

STATUS	BODY	CONDITION
400	`{"message": "Email is required"}`	Missing email
400	`{"message": "Password must be at least 8 characters"}`	Short password
400	`{"message": "Email already in use"}`	Duplicate email

1.2 POST `/api/auth/login`

Description: Authenticate and receive JWT token.

Request Body:

```
{
  "email": "investor@example.com",
  "password": "SecureP@ss123"
}
```

Success Response (200):

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "user": {
    "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "email": "investor@example.com",
    "fullName": "Bhuvan Nagesh"
  }
}
```

Error Response (401):

```
{"message": "Invalid credentials"}
```

1.3 POST `/api/auth/forgot-password`

Description: Send password reset link via email.

Request Body:

```
{"email": "investor@example.com"}
```

Response (200): Always returns success to prevent email enumeration.

```
{"message": "If an account exists with that email, a reset link has been sent"}
```

1.4 POST `/api/auth/reset-password`

Description: Reset password using token from email link.

Request Body:

```
{
  "token": "reset-token-from-email",
  "password": "NewSecureP@ss456"
}
```

Success Response (200):

```
{"message": "Password updated successfully"}
```

1.5 POST `/api/auth/change-password` 🔒

Description: Change password for authenticated user.

Request Body:

```
{
  "currentPassword": "OldP@ss123",
  "newPassword": "NewP@ss456"
}
```

Success Response (200):

```
{"message": "Password changed successfully"}
```

1.6 GET `/api/auth/health`

Description: Health check endpoint.

Response (200):

```
{"status": "ok", "service": "WealthWise Backend"}
```

2. Transaction APIs

2.1 GET `/api/transactions` 🔒

Description: List all transactions for the authenticated user, enriched with scheme metadata.

Response (200):

```
[
  {
    "id": "txn-uuid-1",
    "schemeCode": "119551",
    "fundName": "HDFC Mid-Cap Opportunities Fund - Growth",
    "folioNumber": "WW8A3F21119551",
    "transactionType": "PURCHASE_SIP",
    "amount": 5000.0000,
    "nav": 142.3456,
    "units": 35.123456,
    "transactionDate": "2025-03-15",
    "status": "Completed",
    "source": "Manual",
    "note": null,
    "category": "Equity - Mid Cap",
    "riskLevel": 5,
    "riskLabel": "High",
    "amcName": "HDFC Asset Management Company Limited"
  }
]
```

2.2 POST `/api/transactions` 🔒

Description: Create a new transaction. Automatically creates FIFO lots (BUY) or consumes lots (SELL).

Request Body:

```
{
  "schemeCode": "119551",
  "fundName": "HDFC Mid-Cap Opportunities Fund - Growth",
  "folioNumber": "12345678/90",
  "transactionType": "PURCHASE_SIP",
  "amount": 5000,
  "nav": 142.35,
  "units": null,
  "transactionDate": "2025-03-15",
  "note": "March SIP"
}
```

Accepted Transaction Types: `PURCHASE_LUMPSUM`, `PURCHASE_SIP`, `REDEMPTION`, `SWITCH_IN`, `SWITCH_OUT`, `STP_IN`, `STP_OUT`, `SWP`, `DIVIDEND_PAYOUT`, `DIVIDEND_REINVEST`, `REVERSAL`

Success Response (200): Returns the saved `InvestmentTransaction` object.

2.3 POST `/api/transactions/import-csv` 🔒

Description: Bulk import transactions from a CSV file.

Request: `multipart/form-data` with field `file`

CSV Format (flexible column names):

```
scheme_code,transaction_type,date,amount,nav,units,fund_name,folio,notes
119551,SIP,2025-01-15,5000,138.50,,HDFC Mid-Cap Growth,,Jan SIP
119551,SIP,2025-02-15,5000,140.20,,HDFC Mid-Cap Growth,,Feb SIP
```

Response (200):

```
{
  "imported": 2,
  "failed": 0,
  "skipped": 0,
  "rows": [
    {"row": 2, "status": "OK", "message": "PURCHASE SIP imported", "txId": "uuid-1"},
    {"row": 3, "status": "OK", "message": "PURCHASE SIP imported", "txId": "uuid-2"}
  ]
}
```

---

3. Holdings / Portfolio APIs

3.1 GET `/api/holdings` 🔒

Description: All current fund holdings with gain/loss and XIRR per fund.

Response (200):


```
[
  {
    "schemeCode": "119551",
    "fundName": "HDFC Mid-Cap Opportunities Fund - Growth",
    "amcName": "HDFC AMC",
    "category": "Equity - Mid Cap",
    "planType": "Growth",
    "riskLevel": 5,
    "totalUnits": 245.678900,
    "avgCostNav": 135.4500,
    "investedValue": 33280.50,
    "currentNav": 152.8700,
    "navDate": "2025-04-22",
    "currentValue": 37565.23,
    "gainLoss": 4284.73,
    "gainLossPct": 12.87,
    "xirr": 18.45
  }
]
```

3.2 GET `/api/holdings/summary` 🔒

Description: Portfolio-level aggregated summary.

Response (200):

```
{
  "totalInvested": 500000.00,
  "currentValue": 587500.00,
  "totalGain": 87500.00,
  "gainPct": 17.50,
  "portfolioXirr": 15.23,
  "portfolioRiskScore": 3.8,
  "riskLabel": "Moderately High",
  "fundCount": 8,
  "allocation": {
    "Equity": 420000.00,
    "Debt": 117500.00,
    "Hybrid": 50000.00
  },
  "earliestInvestmentDate": "2023-06-15",
  "holdingYears": 1.85
}
```

3.3 GET `/api/holdings/growth` 🔒

Description: Monthly portfolio value timeline for growth chart.

Response (200):


```
[
  {"month": "2024-01", "investedValue": 100000, "currentValue": 105200},
  {"month": "2024-02", "investedValue": 110000, "currentValue": 118500},
  {"month": "2024-03", "investedValue": 120000, "currentValue": 124800}
]
```

3.4 GET `/api/holdings/risk-profile` 🔒

Description: Advanced risk analytics.

Response (200):

```
{
  "sharpeRatio": 1.42,
  "annualizedVolatility": 14.5,
  "maxDrawdown": -8.3,
  "diversificationScore": 72,
  "concentrationRisk": "Low"
}
```

3.5 GET `/api/holdings/sip-intelligence` 🔒

Description: SIP vs Lumpsum analysis per fund.

Response (200):

```
[
  {
    "schemeCode": "119551",
    "fundName": "HDFC Mid-Cap",
    "sipUnits": 180.50,
    "lumpsumUnits": 65.10,
    "sipAvgNav": 138.20,
    "lumpsumAvgNav": 142.50,
    "sipCostAdvantage": 3.02
  }
]
```

3.6 DELETE `/api/holdings/{schemeCode}` 🔒

Description: Permanently delete all lots and transactions for a specific fund.

Response (200):

```
{"message": "Holding deleted", "schemeCode": "119551"}
```

4. Goal APIs

4.1 GET `/api/goals` 🔒

Description: List all goals with projections and corpus refreshed from linked holdings.

4.2 POST `/api/goals` 🔒

Request Body:

```
{
  "name": "Retirement Fund",
  "goalType": "Retirement",
  "priority": "High",
  "targetAmount": 10000000,
  "targetDate": "2050-04-01",
  "inflationRate": 6.0,
  "expectedReturn": 12.0,
  "monthlyNeed": 25000,
  "currentCorpus": 150000
}
```

4.3 PUT `/api/goals/{id}` 🔒 — Update goal fields

4.4 PATCH `/api/goals/{id}/status` 🔒 — Change status (Active/Achieved/Abandoned)

4.5 DELETE `/api/goals/{id}` 🔒 — Delete goal

4.6 GET `/api/goals/{id}/projection` 🔒 — Deterministic projection

4.7 GET `/api/goals/{id}/monte-carlo` 🔒 — Monte Carlo simulation (5000 runs)

Response:

```
{
  "probability": 73.4,
  "p10": 8500000,
  "p25": 9200000,
  "p50": 10800000,
  "p75": 12500000,
  "p90": 15200000,
  "onTrack": true
}
```

4.8 POST `/api/goals/{id}/link-fund` 🔒 — Link a fund to goal

Request:

```
{
  "schemeCode": "119551",
  "fundName": "HDFC Mid-Cap Growth",
  "folioNumber": "DEFAULT",
  "allocationPct": 60
}
```

4.9 DELETE `/api/goals/{id}/links/{schemeCode}` 🔒 — Unlink fund

4.10 GET `/api/goals/{id}/links` 🔒 — List linked funds

---

5. Scheme & NAV APIs

6.1 GET `/api/schemes/search?q=hdfc+bluechip&limit=15`

Response (200): Array of `SchemeMaster` objects matching the search query.

6.2 GET `/api/schemes/{code}` — Single scheme by AMFI code

6.3 GET `/api/schemes/amcs` — List all AMC names

6.4 GET `/api/schemes/stats` — Total/active scheme counts

6.5 POST `/api/schemes/seed` — Trigger AMFI data re-seed

6.6 GET `/api/nav/{code}/latest` — Latest NAV for a scheme

6.7 GET `/api/nav/{code}/{date}` — NAV on specific date

6.8 GET `/api/nav/{code}/history?days=365` — Historical NAVs

---

7. Alert APIs

7.1 GET `/api/alerts` 🔒 — All alerts (unread first)

7.2 GET `/api/alerts/unread-count` 🔒

Response: `{"count": 3}`

7.3 PATCH `/api/alerts/{id}/read` 🔒 — Mark as read

7.4 DELETE `/api/alerts/{id}` 🔒 — Delete alert

---

8. Profile & Settings APIs

8.1 GET `/api/profile` 🔒 — Get user profile

8.2 PUT `/api/profile` 🔒 — Update profile (fullName, phone)

8.3 GET `/api/settings` 🔒 — Get notification preferences

8.4 PUT `/api/settings` 🔒 — Update notification preferences

9. Goal Engine API (Public)

9.1 POST `/api/learn/analyse`

Description: Standalone goal analysis engine (no auth required). Runs Monte Carlo + deterministic projection + required SIP calculation.

Request Body:

```
{
  "initialPortfolio": 100000,
  "monthlyContribution": 10000,
  "monthlyMean": 0.01,
  "monthlyStdDev": 0.04,
  "months": 240,
  "targetAmount": 10000000,
  "annualInflationRate": 0.06
}
```

Response (200):

```
{
  "monteCarlo": {
    "probability": 68.5,
    "percentiles": {...}
  },
  "deterministic": {
    "projectedValue": 9850000,
    "onTrack": false,
    "shortfall": 150000
  },
  "requiredSip": {
    "monthlySipNeeded": 11200,
    "additionalNeeded": 1200
  }
}
```

HTTP Status Code Reference

CODE	MEANING	WHEN USED
200	OK	Successful request
400	Bad Request	Validation failure, missing fields
401	Unauthorized	Invalid/expired JWT, wrong credentials
403	Forbidden	Accessing another user's resource
404	Not Found	Resource doesn't exist or not owned by user
500	Internal Server Error	Unexpected server exception

DOCUMENT 05

User Manual

05

WealthWise — User Manual

1. System Overview

WealthWise is your personal mutual fund portfolio cockpit. It helps you:

- **Track** all your mutual fund investments in one place
- **Analyse** portfolio performance with XIRR, risk scores, and growth charts
- **Plan** financial goals (retirement, education, house) with Monte Carlo simulations
- **Stay informed** with alerts for SIP due dates, goal milestones, and market events

WealthWise does **not** execute buy or sell orders — it is purely an analytics and planning platform.

System Requirements

REQUIREMENT	SPECIFICATION
Browser	Chrome 90+, Firefox 88+, Safari 15+, Edge 90+
Internet	Required for all features
Device	Desktop or tablet (1024px+ recommended)

2. Getting Started

2.1 Registration

1. Visit the WealthWise landing page
2. Click the **"Get Started"** or **"Sign Up"** button
3. Fill in the registration form:

- **Email Address** — Your primary email (used for login and password recovery) - **Full Name** — Your display name - **Password** — Minimum 8 characters (use a strong, unique password)

1. Click **"Create Account"**
2. You will be automatically logged in and redirected to the Dashboard

> *[Screenshot Placeholder: Signup page with email, name, and password fields]*

2.2 Login

1. Navigate to the **Login** page
2. Enter your registered **email** and **password**
3. Click **"Log In"**
4. On success, you'll be taken to your Dashboard

> *[Screenshot Placeholder: Login page]*

2.3 Forgot Password

1. On the Login page, click **"Forgot Password?"**
2. Enter your registered email address
3. Check your email inbox for a reset link
4. Click the link and enter your new password (minimum 8 characters)
5. Click **"Reset Password"** — you can now log in with the new password

---

3. Dashboard Navigation

After logging in, you'll see the main Dashboard with a **sidebar navigation** on the left.

Navigation Menu

MENU ITEM	ICON	DESCRIPTION
Dashboard	📊	Portfolio overview with summary cards and charts
Investments	💰	View all current holdings by fund
Transactions	📋	Complete transaction history
Portfolio	📈	Advanced analytics, allocation, risk, growth
Goals	🎯	Financial goal planning and tracking
Notifications	🔔	Alerts and system notifications
Settings	⚙️	Notification preferences
Profile	👤	Personal information (name, phone)

Theme Toggle

WealthWise supports **Dark Mode** and **Light Mode**. Use the theme toggle button in the sidebar to switch between themes. The system remembers your preference.

> *[Screenshot Placeholder: Dashboard with sidebar navigation]*

---

4. Feature-by-Feature Usage

4.1 Adding a New Investment / Transaction

Path: Dashboard → Investments → **"+ New Investment"**

1. **Search for Fund**: Start typing the fund name in the search box. WealthWise searches through 45,000+ AMFI-registered mutual fund schemes with fuzzy matching.

- Example: Type "HDFC Blue" to find "HDFC Blue Chip Fund - Growth"

1. **Select Transaction Type**:

- **SIP** — Regular monthly investment - **Lumpsum** — One-time investment - **Redemption** — Selling/withdrawing units - **Switch In/Out** — Transferring between funds - **STP In/Out** — Systematic Transfer Plan - **SWP** — Systematic Withdrawal Plan - **Dividend Reinvest** — IDCW reinvestment

1. **Enter Details**:

- **Transaction Date** — When the transaction occurred - **Amount (₹)** — Investment amount - **NAV** — (Optional) Net Asset Value; auto-fetched if omitted - **Units** — (Optional) Auto-calculated as Amount ÷ NAV

1. **Folio Number** — (Optional) Leave blank for auto-generated folio

1. Click **"Save Transaction"**

> 💡 **Tip:** For BUY transactions, WealthWise automatically creates FIFO cost-basis lots. For SELL transactions, it consumes the oldest lots first and calculates gains.

> *[Screenshot Placeholder: New Investment form with fund search]*

4.2 Importing Transactions via CSV

Path: Dashboard → Investments → **"Import CSV"**

For bulk data entry, you can upload a CSV file with your transaction history.

Supported CSV Columns (flexible names):

COLUMN	ALIASES ACCEPTED
Scheme Code	`scheme_code`, `amfi_code`, `schemecode`, `code`
Transaction Type	`transaction_type`, `type`, `txn_type`, `txtype`
Date	`date`, `transaction_date`, `txn_date`
Amount	`amount`
NAV	`nav`, `purchase_nav`, `allotment_nav`
Units	`units`
Fund Name	`fund_name`, `scheme_name`, `fund`
Folio	`folio_number`, `folio`
Notes	`note`, `notes`, `remarks`

Supported Date Formats: `yyyy-MM-dd`, `dd-MM-yyyy`, `dd/MM/yyyy`, `MM/dd/yyyy`, `d-MMM-yyyy`

Steps:

1. Prepare your CSV file with at least `scheme_code`, `transaction_type`, and `date` columns
2. Click **"Upload CSV"** and select your file
3. WealthWise will process each row and show results:

- ✅ **OK** — Successfully imported - ❌ **FAILED** — Error (with specific reason) - ⏭️ **SKIPPED** — Empty rows

> *[Screenshot Placeholder: CSV import results screen]*

4.3 Viewing Your Portfolio

Path: Dashboard → **Portfolio**

The Portfolio page provides comprehensive analytics:

Summary Cards

- **Total Invested** — Sum of all purchase transactions
- **Current Value** — Market value based on latest NAVs
- **Total Gain/Loss** — Absolute and percentage
- **Portfolio XIRR** — Annualized returns accounting for timing
- **Risk Score** — Weighted average (1-6 scale) with label

Asset Allocation Chart

- Pie chart showing distribution across: Equity, Debt, Hybrid, Commodity, Other
- Helps assess diversification

Holdings Table

- Each fund showing: units, avg cost, invested value, current value, gain/loss %, XIRR

- Sortable columns

Growth Timeline

- Line chart showing monthly portfolio value vs invested value over time

Risk Profile

- Sharpe Ratio, Volatility, Maximum Drawdown, Diversification Score

> *[Screenshot Placeholder: Portfolio page with charts and holdings table]*

4.4 Setting Financial Goals

Path: Dashboard → Goals → **"+ New Goal"**

1. **Select Goal Type:**

- 🏠 House Purchase - 🎓 Education - 🌴 Retirement - 🚗 Vehicle - 🛟 Emergency Fund - ✨ Custom

1. **Fill Goal Details:**

- **Goal Name** — e.g., "Daughter's Education" - **Target Amount (₹)** — The amount you want to accumulate - **Target Date** — When you need the money - **Priority** — High / Medium / Low - **Monthly SIP** — How much you plan to invest monthly - **Current Corpus** — How much you've already saved - **Inflation Rate** — Expected annual inflation (default: 6%) - **Expected Return** — Expected annual portfolio return (default: 12%)

1. Click **"Create Goal"**

> *[Screenshot Placeholder: Goal creation wizard]*

4.5 Analysing Goals

Each goal card on the Goals page shows:

- **Progress bar** — Current corpus vs inflation-adjusted target
- **On Track / Off Track** indicator
- **Required Monthly SIP** — What you need to invest to hit the target

Click **"Analyse"** on any goal card to open the detailed analysis modal:

Deterministic Projection

- Compound growth projection showing year-by-year values
- Shows whether current SIP + corpus will reach target

Monte Carlo Simulation

- Runs 5,000 random simulations with varying market returns
- Shows **probability of success** (e.g., "73.4% chance of achieving goal")
- Percentile breakdown: P10, P25, P50 (median), P75, P90

Linking Funds to Goals

- Click **"Link Fund"** to associate specific mutual fund holdings with a goal
- Set allocation percentage (e.g., 60% of HDFC Mid-Cap linked to Retirement goal)
- Goal corpus auto-updates from linked fund values

> *[Screenshot Placeholder: Goal analysis modal with Monte Carlo chart]*

4.6 Viewing Notifications

Path: Dashboard → **Notifications** (🔔)

The notification bell in the sidebar shows unread count. The Notifications page lists all alerts:

- **Info** (blue) — Informational updates
- **Watch** (yellow) — Items needing attention
- **Action** (red) — Immediate action required

Actions:

- Click an alert to mark as read
- Click the dismiss button to delete

4.7 Configuring Settings

Path: Dashboard → **Settings** (⚙️)

Toggle notification preferences:

SETTING	IN-APP	EMAIL
SIP Due Reminders	✅	▪
Goal Milestones	✅	▪
Daily Digest	—	▪
Market Alerts	✅	—
Missed SIP Alert	✅	—

4.8 Managing Profile

Path: Click your avatar → **Profile**

Update your personal information:

- Full Name
- Phone Number

---

5. Navigation Quick Reference

```
Landing Page (/)
├── Login (/login)
├── Signup (/signup)
├── Forgot Password (/forgot-password)
└── Reset Password (/reset-password)

Dashboard (/dashboard)  ← Requires login
├── Investments (/dashboard/investments)
│   ├── New Investment (/dashboard/investments/new)
│   └── Import CSV (/dashboard/investments/import)
├── Transactions (/dashboard/transactions)
├── Portfolio (/dashboard/portfolio)
├── Goals (/dashboard/goals)
│   └── New Goal (/dashboard/goals/new)
├── Notifications (/dashboard/notifications)
├── Settings (/dashboard/settings)
└── Profile (/profile)
```

6. Frequently Asked Questions (FAQ)

Q: Does WealthWise execute actual buy/sell orders?

A: No. WealthWise is an analytics and planning platform. It tracks and analyses your investments but does not execute transactions with fund houses.

Q: Where does WealthWise get NAV data?

A: NAV data is sourced from the **AMFI (Association of Mutual Funds in India)** official API, which publishes daily NAVs for all registered mutual fund schemes in India.

Q: How is XIRR different from CAGR?

A: CAGR assumes a single lumpsum investment. **XIRR** accounts for multiple cash flows at different dates (SIPs, additional purchases, partial redemptions), giving a more accurate annualized return for real-world portfolios.

Q: How does FIFO cost-basis work?

A: When you sell (redeem) units, WealthWise automatically sells the **oldest purchased lots first** (First-In-First-Out). This is the standard method for computing cost basis and gains.

Q: What is Monte Carlo simulation?

A: It's a statistical technique that runs 5,000 random simulations of your portfolio growth, each with different market return scenarios. The result is a **probability** of achieving your

financial goal (e.g., "73% chance of reaching ₹1 Crore by 2045").

Q: Can I import data from my CAS (Consolidated Account Statement)?

A: Yes, WealthWise supports CSV import. Export your CAS data as CSV from CAMS/KFintech and upload it. The system intelligently maps different column name formats.

Q: Is my data secure?

A: Yes. Passwords are hashed with BCrypt (never stored in plain text). All API communications use HTTPS. Authentication uses JWT tokens with configurable expiry. Each user can only access their own data.

Q: Can I use WealthWise on my phone?

A: WealthWise is optimized for desktop and tablet browsers (1024px+ width). Mobile support with responsive design is planned for future versions.

Q: How do I delete all data for a specific fund?

A: Go to your Holdings page, find the fund, and click the delete/remove button. This permanently removes all transactions and lots for that fund.

---

7. Glossary

TERM	DEFINITION
NAV	Net Asset Value — the per-unit market price of a mutual fund
SIP	Systematic Investment Plan — regular periodic investments
XIRR	Extended Internal Rate of Return — annualized return for irregular cash flows
FIFO	First-In, First-Out — method of lot selection when selling
STCG	Short-Term Capital Gains — profits on holdings < 1 year
LTCG	Long-Term Capital Gains — profits on holdings ≥ 1 year
AMC	Asset Management Company — the fund house managing the scheme
AMFI	Association of Mutual Funds in India — industry regulatory body
Folio	Unique account number for your holdings with an AMC
Lot	A specific purchase of units at a specific date and NAV
Corpus	Total accumulated amount in an investment or goal
Sharpe Ratio	Risk-adjusted return metric (higher is better)
Drawdown	Peak-to-trough decline in portfolio value

---

8. Support & Contact

For technical issues, feature requests, or feedback:

- **Repository:** GitHub — WealthWise Project
- **Email:** Support team via project maintainers

DOCUMENT 06

🧪 Testing Documentation

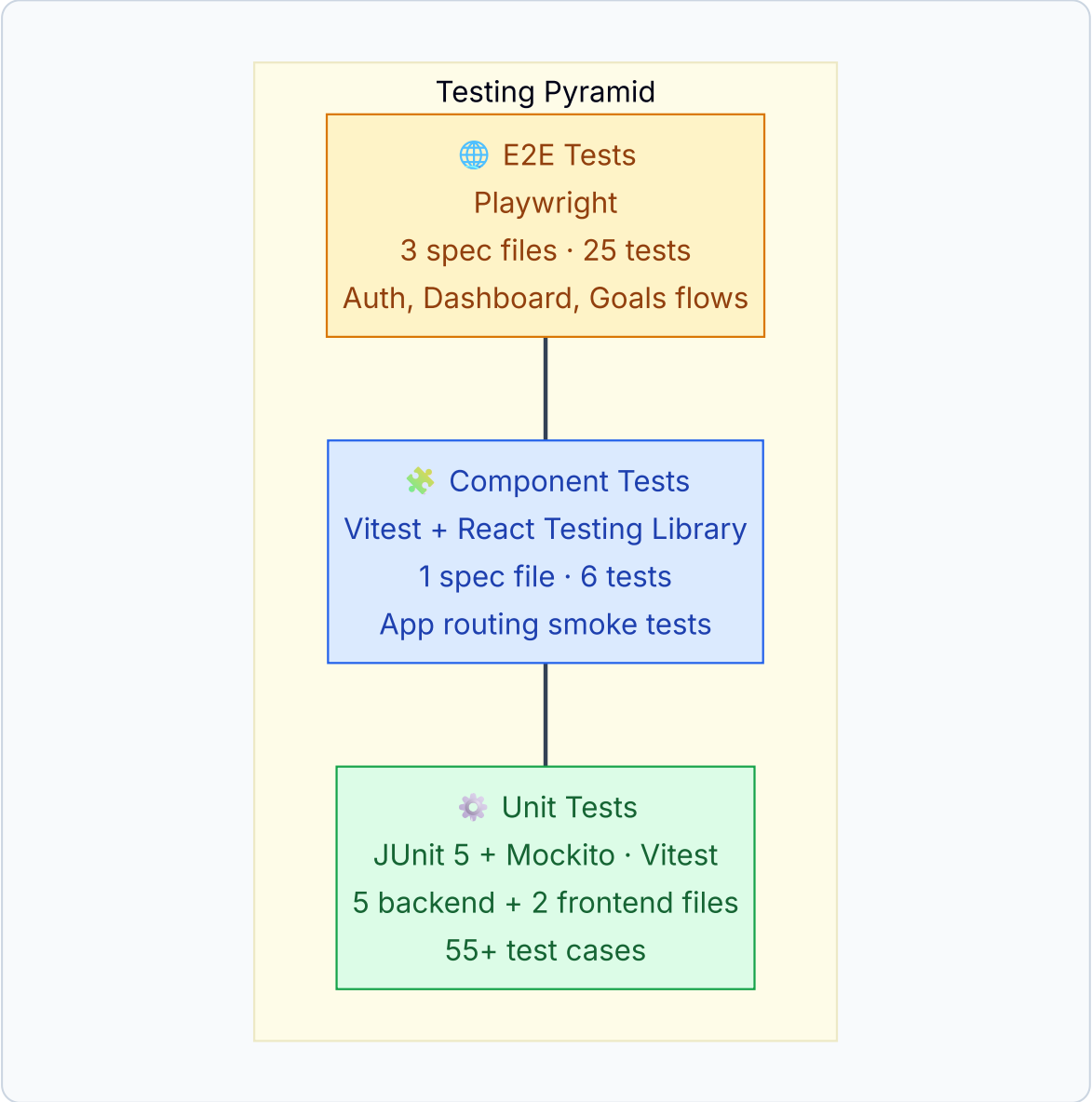
06

🧪 WealthWise — Testing Documentation

Document Version: 1.0 **Date:** April 2026

6.1 Testing Strategy Overview

WealthWise employs a **three-tier testing pyramid** to ensure system reliability at every level:



LAYER	FRAMEWORK	SCOPE	FILES	TEST COUNT
Unit Tests (Backend)	JUnit 5 + Mockito + AssertJ	Service logic, Controller endpoints	5	~40
Unit Tests (Frontend)	Vitest	Pure utility functions	2	~30
Component Tests	Vitest + React Testing Library	React routing, component render	1	6
E2E Tests	Playwright	Full browser UI flows	3	25

6.2 Backend Unit Tests

Test Framework Stack

TECHNOLOGY	PURPOSE
JUnit 5	Test runner and assertions
Mockito	Mock dependencies (repositories, services)
AssertJ	Fluent assertion library
MockMvc	Standalone Spring MVC test (no full context)
H2 Database	In-memory DB for integration tests

6.2.1 AuthServiceTest — 9 Tests

File: `backend/src/test/java/com/wealthwise/service/AuthServiceTest.java` **Pattern:** Pure unit test with `@ExtendWith(MockitoExtension.class)` — no Spring context.

TEST CASE	ASSERTION
`signup_success`	New user persisted, password BCrypt-hashed (not plain text), email stored
`signup_emailNormalisedToLowercase`	`BOB@EXAMPLE.COM` → stored as `bob@example.com`
`signup_duplicateEmail_throws`	Throws `RuntimeException("already exists")`, `save()` never called
`login_success`	Valid credentials return `AppUser` object
`login_wrongPassword_throws`	Wrong password throws `RuntimeException("Invalid email or password")`
`login_unknownEmail_throws`	Non-existent email throws `RuntimeException`
`generateToken_returnsJwt`	Token is non-blank, has 3 dot-separated segments (JWT format)
`changePassword_success`	New password BCrypt-verified, `save()` called
`changePassword_wrongCurrentPassword_throws`	Exception thrown, `save()` never called

Key technique: Uses `ReflectionTestUtils` to inject real `BCryptPasswordEncoder` and fake JWT secret into the service without Spring context.

6.2.2 AuthControllerTest — 10 Tests

File: `backend/src/test/java/com/wealthwise/controller/AuthControllerTest.java` **Pattern:** Standalone `MockMvc` via `MockMvcBuilders.standaloneSetup()` — avoids full `@WebMvcTest` context.

TEST CASE	HTTP	EXPECTED
`health_returns200`	`GET /api/auth/health`	200 + `{"status":"ok"}`
`signup_success_returns200WithToken`	`POST /api/auth/signup`	200 + `{token, user}`
`signup_blankEmail_returns400`	`POST /api/auth/signup`	400 + `"Email is required"`
`signup_shortPassword_returns400`	`POST /api/auth/signup`	400 + `"Password must be at least 8 characters"`
`signup_duplicateEmail_returns400`	`POST /api/auth/signup`	400 + `"already exists"`
`login_success_returns200WithToken`	`POST /api/auth/login`	200 + `{token, user}`
`login_wrongCredentials_returns401`	`POST /api/auth/login`	401 + `"Invalid email or password"`
`login_blankEmail_returns400`	`POST /api/auth/login`	400
`login_blankPassword_returns400`	`POST /api/auth/login`	400
`forgotPassword_returns200Always`	`POST /api/auth/forgot-password`	200 always (prevents email enumeration)

6.2.3 XirrServiceTest — 11 Tests

File: `backend/src/test/java/com/wealthwise/service/XirrServiceTest.java` **Pattern:** Pure unit test — no mocks, no Spring context. Tests the Newton-Raphson numerical solver.

TEST CASE	ASSERTION
`calculateXirr_nullList_returnsNull`	Null input → null
`calculateXirr_singleFlow_returnsNull`	Need at least 2 cash flows
`calculateXirr_allNegative_returnsNull`	All outflows → null
`calculateXirr_allPositive_returnsNull`	All inflows → null
`calculateXirr_sameDayFlows_returnsNull`	Zero-day span → null
`calculateXirr_lumpsum10Pct`	₹10K→₹11K in 1yr ≈ 10% XIRR
`calculateXirr_monthlySips`	12 monthly SIPs → positive annualized rate
`calculateXirr_resultClamping`	Result clamped to [-0.99, 5.0]
`absoluteReturn_correctFormula`	(12500-10000)/10000 = 0.25
`absoluteReturn_zeroInvested_returnsNull`	Division by zero guarded
`cagr_doubling_threeYears`	2x in 3yrs ≈ 26% CAGR

6.2.4 GoalProjectionServiceTest — 11 Tests

File: `backend/src/test/java/com/wealthwise/service/GoalProjectionServiceTest.java` **Pattern:** Pure unit test — validates deterministic and Monte Carlo math.

Deterministic projection tests:

TEST CASE	ASSERTION
`project_pastDueGoal_returnsZeroed`	Past-due → `onTrack=false`, `monthsRemaining=0`
`project_onTrack_whenProjectedExceedsTarget`	₹10L corpus vs ₹2L target → `onTrack=true`
`project_noCorpusNoSip_gapEqualsTarget`	No money → gap = full target amount
`project_resultContainsAllRequiredKeys`	Contains: projectedValue, gap, onTrack, requiredSip, monthsRemaining, fvCorpus, fvSip
`project_futureGoal_positiveMonthsRemaining`	Future date → months > 0
`project_zeroCorpus_fvCorpusIsZero`	Zero corpus → fvCorpus = 0
`project_nullExpectedReturn_defaultsTo12Pct`	Null rate → defaults to 12%, no exception

Monte Carlo simulation tests:

TEST CASE	ASSERTION
`monteCarlo_pastDueGoal_zeroProb`	Past-due → probability = 0%
`monteCarlo_resultContainsAllKeys`	Contains: probability, p5, p50, p95
`monteCarlo_probabilityIsInValidRange`	Result ∈ [0, 100]
`monteCarlo_percentilesAreOrdered`	p5 ≤ p50 ≤ p95
`monteCarlo_wellFundedGoal_highProbability`	20x over-funded → probability > 90%
`monteCarlo_unreachableGoal_lowProbability`	₹100Cr target with ₹100/mo → probability < 1%

6.2.5 HoldingsControllerTest

File: `backend/src/test/java/com/wealthwise/controller/HoldingsControllerTest.java` Tests portfolio summary, holdings list, and XIRR calculation endpoints using MockMvc.

6.3 Frontend Unit Tests

Test Framework Stack

TECHNOLOGY	PURPOSE
Vitest 3.1	Test runner (Vite-native, blazing fast)
React Testing Library	DOM testing for React components
jsdom	Browser environment simulation

6.3.1 goalHelpers.test.js — 22 Tests

File: `frontend/src/lib/__tests__/goalHelpers.test.js` Tests all pure utility functions used in goal planning calculations.

FUNCTION	TESTS	KEY ASSERTIONS
`GOAL_TYPES`	3	Contains 8 types, each has required fields (type, icon, label, inflationRate, suggestedYears, expectedReturn)
`getAllocationSuggestion()`	7	>10yr→Aggressive(80/20), >5yr→Balanced(60/40), >3yr→Conservative(40/60), >1yr→Preservation(20/80), ≤1yr→Liquid(0/100)
`futureValue()`	6	FV=100K×1.12^10≈310K, 0yr→PV unchanged, null→0
`yearsUntil()`	4	null→0, +1yr≈1.0, +5yr≈5.0, past→0
`recommendedSIP()`	4	₹10L target in 10yr@12%≈₹4,347/mo, 0yr→FV, shorter horizon→higher SIP
`formatINR()`	10	null→"—", ₹1Cr→"₹1.00 Cr", ₹1L→"₹1.00 L", ₹5K→"₹5.0K", ₹500→"₹500"

6.3.2 mfApi.test.js — ~8 Tests

File: `frontend/src/lib/__tests__/mfApi.test.js` Tests the AMFI mutual fund API integration helpers (search, NAV lookup, scheme metadata).

6.4 Component Tests

6.4.1 App.test.jsx — 6 Tests

File: `frontend/src/__tests__/App.test.jsx` **Pattern:** Uses `MemoryRouter` to test React Router routes without a real browser. All page components are mocked to lightweight stubs for fast execution.

TEST CASE	ASSERTION
`renders LandingPage at /`	`page-landing` testid visible
`renders LoginPage at /login`	`page-login` testid visible
`renders SignupPage at /signup`	`page-signup` testid visible
`renders ForgotPasswordPage at /forgot-password`	`page-forgot` testid visible
`redirects /dashboard to /login (unauthenticated)`	Protected route → login redirect
`redirects unknown routes to / (catch-all)`	Unknown path → landing page

Key technique: Mocks `useAuth()` to return `user: null` (unauthenticated), verifying the auth guard `ProtectedRoute` redirects correctly.

6.5 End-to-End (E2E) Tests

Playwright Configuration

File: `frontend/playwright.config.js`

SETTING	VALUE
Test Directory	`frontend/e2e/`
Base URL	`http://localhost:5173`
Timeout	30 seconds per test
Browsers	Chromium, Firefox, WebKit, Mobile Chrome
Auto-start	Vite dev server starts before tests
Screenshots	Only on failure
Tracing	On first retry
Retries	0 (local), 2 (CI)

6.5.1 auth.spec.js — 13 Tests

File: `frontend/e2e/auth.spec.js`

Landing Page (2 tests):

- Page loads without crashing
- CTA button or navigation link is visible

Login Page (6 tests):

- Email and password fields visible
- Submit button visible
- Form fields accept text input
- Wrong credentials show error (backend required)
- Link to signup page exists
- Forgot password link exists

Signup Page (5 tests):

- Registration form displays
- Password field present
- Submit button present
- Form fields accept input
- Link back to login exists

Forgot Password Page (3 tests):

- Form displays with email field
- Submit button present
- Email submission shows result

6.5.2 dashboard.spec.js — 6 Tests

File: `frontend/e2e/dashboard.spec.js`

Auth Guard (8 protected routes tested): All protected routes redirect to `/login` when unauthenticated:

- `/dashboard`, `/dashboard/investments`, `/dashboard/transactions`
- `/dashboard/portfolio`, `/dashboard/goals`, `/dashboard/notifications`
- `/dashboard/settings`, `/profile`

404 Handling (2 tests):

- Unknown URL redirects to landing page
- Deeply nested unknown URL redirects to `/`

Navigation (2 tests):

- Login link navigates to `/login`
- Signup link navigates to `/signup`

6.5.3 goals.spec.js — 5 Tests

File: `frontend/e2e/goals.spec.js`

Auth Guard (2 tests):

- `/dashboard/goals` redirects to login when unauthenticated
- `/dashboard/goals/new` redirects to login when unauthenticated

Goals Page UI (2 tests, with injected fake auth):

- Goals page loads without crashing
- Shows create goal button or empty state

New Goal Flow (2 tests):

- Navigation to `/dashboard/goals/new` shows the goal wizard
- New goal page has form inputs

Key technique: Uses `page.addInitScript()` to inject fake localStorage auth tokens, bypassing frontend auth without a real backend.

6.6 Running Tests

Backend Tests

```
# Run all backend tests
cd backend
./mvnw test

# Run specific test class
./mvnw test -Dtest=AuthServiceTest

# Run with verbose output
./mvnw test -Dtest=XirrServiceTest -pl . -Dsurefire.useFile=false
```

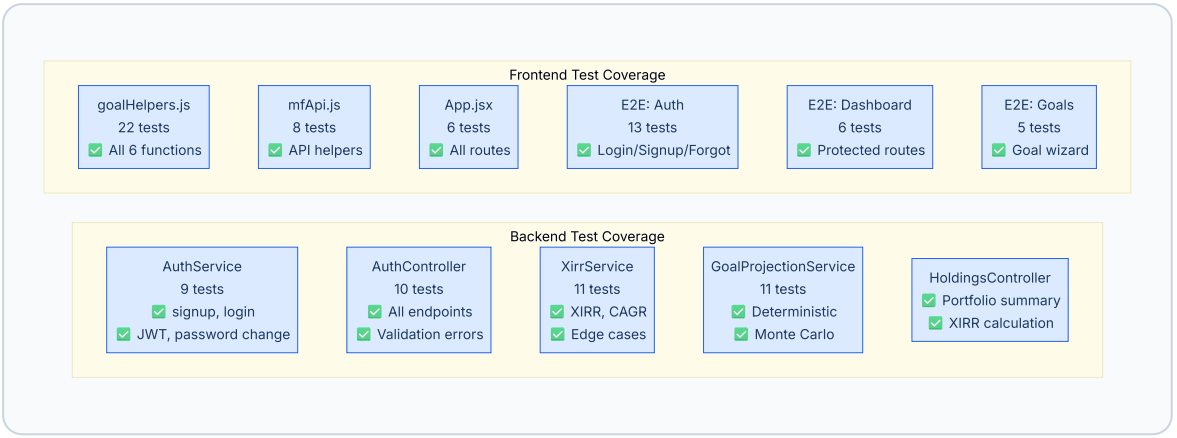
Frontend Unit Tests

```
# Run all Vitest tests
cd frontend
npx vitest run

# Run in watch mode
npx vitest

# Run with coverage
npx vitest run --coverage
```

E2E Tests

```
# Run all Playwright tests
cd frontend
npx playwright test

# Run with UI mode (interactive)
npx playwright test --ui

# Run specific spec file
npx playwright test e2e/auth.spec.js

# Run on specific browser
npx playwright test --project=chromium

# Generate HTML report
npx playwright show-report
```

6.7 Test Coverage Summary



MODULE	TESTS	COVERAGE FOCUS
Authentication (Backend)	19	Signup, login, JWT, password change, validation
XIRR Calculator	11	Newton-Raphson solver, edge cases, clamping
Goal Projection	11	Deterministic math, Monte Carlo statistics
Holdings	5+	Portfolio aggregation, XIRR
Goal Helpers (Frontend)	22	FV, SIP, allocation, formatting
MF API (Frontend)	8	Scheme search, NAV lookup
App Routing	6	Route rendering, auth guard
E2E Auth Flows	13	Landing, login, signup, forgot password
E2E Dashboard	6	Protected routes, 404 handling
E2E Goals	5	Goal creation wizard
Total	**~106**	

Document Version: 1.0 | Last Updated: April 2026

WealthWise — Deployment Guide

1. Prerequisites

1.1 Software Requirements

SOFTWARE	MINIMUM VERSION	RECOMMENDED VERSION	PURPOSE
Java JDK	17	Eclipse Temurin 17.0.x	Backend compilation and runtime
Apache Maven	3.8.x	3.8.7+	Backend dependency management and build
Node.js	18.x	22.x LTS	Frontend build tooling (Vite)
npm	9.x	10.x+	Frontend package management
PostgreSQL	14.x	15.x	Database (or use Supabase cloud)
Git	2.30+	Latest	Source code management
Docker	20.10+	24.x+	Container builds (production)

1.2 Accounts Required

SERVICE	PURPOSE	URL
Supabase	Managed PostgreSQL hosting	https://supabase.com
Render	Cloud deployment (backend + frontend)	https://render.com
Gmail	SMTP email for OTP delivery	Requires App Password
GitHub	Source code repository	https://github.com

1.3 Verify Installation

```
# Verify Java
java -version
# Expected: openjdk version "17.0.x"

# Verify Maven
mvn -version
# Expected: Apache Maven 3.8.x

# Verify Node.js
node -v
# Expected: v22.x.x

# Verify npm
npm -v
# Expected: 10.x.x

# Verify Docker (if building containers)
docker --version
# Expected: Docker version 24.x.x
```

2. Environment Configuration

2.1 Backend Environment Variables

The backend reads configuration from environment variables (not committed to version control). A template is provided at `backend/.env.local.example`.

VARIABLE	REQUIRED	DESCRIPTION	EXAMPLE VALUE
`DB_URL`	✅	JDBC connection string to PostgreSQL database. Must include SSL mode for Supabase.	`jdbc:postgresql://db.xxx.supabase.co:5432/postgres?sslmode=require`
`DB_USERNAME`	✅	Database username	`postgres`
`DB_PASSWORD`	✅	Database password	`your-secure-password`
`JWT_SECRET`	✅	HMAC-SHA256 signing key for JWT tokens. Must be ≥32 characters. Also used to derive AES-256 key for PAN encryption.	`WealthWise$ecretKey2026!SuperSecureRandomString#MF`
`MAIL_USERNAME`	✅	Gmail address for sending OTP emails	`yourname@gmail.com`
`MAIL_PASSWORD`	✅	Gmail App Password (not regular password). Generate at https://myaccount.google.com/apppasswords	`abcd efgh ijkl mnop`
`CORS_ALLOWED_ORIGINS`	⚠️	Comma-separated list of allowed frontend origins. Defaults to localhost if not set.	`http://localhost:5173,https://your-frontend.onrender.com`
`SPRING_PROFILES_ACTIVE`	⚠️	Spring profile. Use `prod` for production deployment on Render.	`prod`
`PORT`	⚠️	Server port. Render sets this automatically. Default: 8080	`8080`

2.2 Frontend Environment Variables

The frontend uses Vite's `import.meta.env` system. Variables are read from `.env` files in the `project/` directory.

VARIABLE	FILE	DESCRIPTION	EXAMPLE VALUE
`VITE_API_BASE_URL`	`.env` (dev), `.env.production` (prod)	Backend API base URL (no trailing slash)	`http://localhost:8080` (dev) / `https://wealthwise-backend-zv5r.onrender.com` (prod)

2.3 Security Notes

> ⚠️ ***CRITICAL:*** *Never commit secrets to version control. The following files are in* `.gitignore`*:*
>
> - `backend/src/main/resources/application.properties`
> - `backend/src/main/resources/application-local.properties`
> - `project/.env`
> - `project/.env.local`
> - `project/.env.*.local`

3. Local Development Setup

3.1 Clone the Repository

```
git clone https://github.com/your-username/Wealthwise.git
cd Wealthwise
```

3.2 Backend Setup

Step 1: Configure Environment Variables

```
# Copy the example environment file
cd backend
copy .env.local.example .env.local
```

Edit `backend/.env.local` with your actual values:

```
DB_URL=jdbc:postgresql://db.xxx.supabase.co:5432/postgres?sslmode=require
DB_USERNAME=postgres
DB_PASSWORD=your-password
JWT_SECRET=your-32-character-minimum-secret-key-here
MAIL_USERNAME=yourname@gmail.com
MAIL_PASSWORD=your-app-password
CORS_ALLOWED_ORIGINS=http://localhost:5173,http://localhost:3000
```

Step 2: Create application.properties

Create `backend/src/main/resources/application.properties`:

```
# ── Database ──────────────────────────────────
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
spring.datasource.driver-class-name=org.postgresql.Driver

# ── JPA / Hibernate ───────────────────────────
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.properties.hibernate.format_sql=true

# ── JWT ───────────────────────────────────────
app.jwt.secret=${JWT_SECRET}
app.jwt.expiration-ms=86400000

# ── Mail (Gmail SMTP) ─────────────────────────
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=${MAIL_USERNAME}
spring.mail.password=${MAIL_PASSWORD}
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true

# ── CORS ──────────────────────────────────────
cors.allowed.origins=${CORS_ALLOWED_ORIGINS:http://localhost:5173}

# ── File Upload ───────────────────────────────
spring.servlet.multipart.max-file-size=10MB
spring.servlet.multipart.max-request-size=10MB

# ── Server ────────────────────────────────────
server.port=${PORT:8080}
```

Step 3: Set Environment Variables (Windows PowerShell)

```
$env:DB_URL = "jdbc:postgresql://db.xxx.supabase.co:5432/postgres?sslmode=require"
$env:DB_USERNAME = "postgres"
$env:DB_PASSWORD = "your-password"
$env:JWT_SECRET = "WealthWise-SecretKey2026-SuperSecure"
$env:MAIL_USERNAME = "yourname@gmail.com"
$env:MAIL_PASSWORD = "your-app-password"
$env:CORS_ALLOWED_ORIGINS = "http://localhost:5173"
```

Step 4: Build and Run

```
cd backend

# Download dependencies (first time only)
mvn dependency:resolve

# Build the project (skip tests for speed)
mvn clean package -DskipTests

# Run the backend
mvn spring-boot:run

# Or run the JAR directly
java -jar target/wealthwise-backend-0.0.1-SNAPSHOT.jar
```

The backend will start on `http://localhost:8080`. Verify with:

```
curl http://localhost:8080/api/auth/health
# Expected: {"status":"UP"}
```

3.3 Frontend Setup

```
cd project

# Install dependencies
npm install

# Create .env with local backend URL
echo "VITE_API_BASE_URL=http://localhost:8080" > .env

# Start development server with HMR
npm run dev
```

The frontend will start on `http://localhost:5173`.

3.4 Database Setup

If using **Supabase** (recommended):

1. Create a project at https://supabase.com
2. Go to Settings → Database → Connection String → URI
3. Use the JDBC format: `jdbc:postgresql://db.xxx.supabase.co:5432/postgres?sslmode=require`
4. Spring Boot's `ddl-auto=update` will create tables automatically on first run

If using **local PostgreSQL**:

```
CREATE DATABASE wealthwise;
-- Tables are auto-created by JPA/Hibernate on first run
```

3.5 Seed Scheme Data

After the backend is running, seed the scheme master database:

```
# This fetches ~45,000 schemes from AMFI and seeds the scheme_master table
curl -X POST http://localhost:8080/api/schemes/seed
```

> ***Note:*** *This operation takes 30–60 seconds and should be run once after initial setup.*

---

4. Production Deployment (Render)

4.1 Repository Structure for Render

Render expects the following structure (matching the existing `render.yaml`):

```
Wealthwise/
├── backend/
│   ├── Dockerfile          ← Render builds the backend from this
│   ├── pom.xml
│   └── src/
├── project/
│   ├── package.json        ← Render runs `npm ci && npm run build`
│   └── src/
└── render.yaml             ← Infrastructure-as-code deployment spec
```

4.2 Deploy Backend (Docker Web Service)

1. **Connect Repository:** In Render dashboard → New → Web Service → Connect GitHub repo
2. **Configuration:**
   - **Name:** `wealthwise-backend`
   - **Runtime:** Docker
   - **Dockerfile Path:** `./backend/Dockerfile`
   - **Docker Context:** `./backend`
   - **Plan:** Free
   - **Health Check Path:** `/api/auth/health`
3. **Environment Variables:** Set in Render's Environment tab:

KEY	VALUE
`SPRING_PROFILES_ACTIVE`	`prod`
`PORT`	`8080`
`DB_URL`	`jdbc:postgresql://db.xxx.supabase.co:5432/postgres?sslmode=require`
`DB_USERNAME`	`postgres`
`DB_PASSWORD`	`<your supabase password>`
`JWT_SECRET`	`<min 32-char random string>`
`MAIL_USERNAME`	`yourname@gmail.com`
`MAIL_PASSWORD`	`<gmail app password>`
`CORS_ALLOWED_ORIGINS`	`https://your-frontend.onrender.com`

4. **Docker Build Process:**

```
# Stage 1: Build (Maven + JDK 17)
FROM maven:3.8.7-eclipse-temurin-17 AS build
COPY pom.xml .
RUN mvn dependency:go-offline -B  # Cached layer
COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2: Run (JRE 17 only — smaller image)
FROM eclipse-temurin:17-jre-jammy
COPY --from=build /app/target/*.jar app.jar
ENTRYPOINT ["java", "-XX:TieredStopAtLevel=1", "-XX:MaxRAMPercentage=70.0",
            "-XX:+UseSerialGC", "-Xss512k", "-jar", "app.jar"]
```

4.3 Deploy Frontend (Static Site)

1. **In Render dashboard:** New → Static Site → Connect GitHub repo
2. **Configuration:**
   - **Name:** `wealthwise-frontend`
   - **Build Command:** `cd project && npm ci && npm run build`
   - **Publish Directory:** `./project/dist`
3. **Rewrite Rule:** Add a rewrite `/* → /index.html` (SPA routing support)
4. **Environment Variables:**

KEY	VALUE
`VITE_API_BASE_URL`	`https://wealthwise-backend-zv5r.onrender.com`

4.4 render.yaml (Infrastructure as Code)

The project includes a `render.yaml` file that defines both services declaratively:

```
services:
  - type: web
    name: wealthwise-backend
    runtime: docker
    dockerfilePath: ./backend/Dockerfile
    dockerContext: ./backend
    plan: free
    healthCheckPath: /api/auth/health
    envVars:
      - key: SPRING_PROFILES_ACTIVE
        value: prod
      - key: PORT
        value: 8080

  - type: web
    name: wealthwise-frontend
    runtime: static
    staticPublishPath: ./project/dist
    buildCommand: cd project && npm ci && npm run build
    plan: free
    routes:
      - type: rewrite
        source: /*
        destination: /index.html
```

---

5. Cold-Start Warmup Architecture

Render's free tier spins down containers after 15 minutes of inactivity. WealthWise handles this with a multi-layer warmup strategy:

5.1 Frontend Warmup Overlay

On application load, `BackendWarmupContext` immediately pings `GET /api/auth/health`:

1. **If backend responds in <5s** → `isWarm = true` → no overlay shown
2. **If backend is cold** → Show animated overlay: "Waking up the server... Xs"
3. **Poll every 3s** until backend responds or 90s timeout
4. **After warm:** Keep-alive ping fires every 9 minutes to prevent re-spin-down

5.2 JVM Tuning for Fast Cold Start

```
-XX:TieredStopAtLevel=1   → Skip C2 JIT (saves ~20s startup)
-XX:MaxRAMPercentage=70.0 → Cap heap at 70% of 512 MB
-XX:+UseSerialGC          → Lower memory footprint than G1
-Xss512k                  → Smaller thread stacks
```

---

6. Build Commands Reference

ACTION	COMMAND	DIRECTORY
Backend — Install dependencies	`mvn dependency:resolve`	`backend/`
Backend — Build JAR	`mvn clean package -DskipTests`	`backend/`
Backend — Run (Maven)	`mvn spring-boot:run`	`backend/`
Backend — Run (JAR)	`java -jar target/wealthwise-backend-0.0.1-SNAPSHOT.jar`	`backend/`
Backend — Docker Build	`docker build -t wealthwise-backend .`	`backend/`
Backend — Docker Run	`docker run -p 8080:8080 --env-file .env.local wealthwise-backend`	`backend/`
Frontend — Install	`npm install`	`project/`
Frontend — Dev Server	`npm run dev`	`project/`
Frontend — Production Build	`npm run build`	`project/`
Frontend — Preview Build	`npm run preview`	`project/`
Frontend — Lint	`npm run lint`	`project/`
Scheme Seed	`curl -X POST http://localhost:8080/api/schemes/seed`	Any

---

7. Common Issues & Troubleshooting

Issue 1: Backend fails to start — "Failed to configure DataSource"

Cause: Database connection environment variables not set or incorrect.
Fix: Ensure `DB_URL`, `DB_USERNAME`, and `DB_PASSWORD` are set as system environment variables before running the app. Verify the Supabase connection string includes `?sslmode=require`.

Issue 2: "Could not resolve placeholder 'app.jwt.secret'"

Cause: `JWT_SECRET` environment variable not set.
Fix: Set `JWT_SECRET` with a ≥32-character string.

Issue 3: Frontend shows "Network Error" on all API calls

Cause: Backend not running or CORS not configured.
Fix:

1. Verify backend is running: `curl http://localhost:8080/api/auth/health`
2. Ensure `CORS_ALLOWED_ORIGINS` includes the frontend URL

Issue 4: CAS PDF import returns "Valid PDF file is required"

Cause: File is not a valid PDF or content-type header is wrong.
Fix: Ensure the uploaded file is a genuine PDF (not renamed).

Issue 5: "Duplicate key value violates unique constraint nav_history"

Cause: Concurrent NAV history insert race condition.
Fix: The system uses `INSERT ON CONFLICT DO NOTHING` — this should not occur. If it does, verify the nav_history unique constraint exists: `UNIQUE(amfi_code, nav_date)`.

Issue 6: Render backend takes >90 seconds to start

Cause: Free tier cold start with large dependency tree.
Fix: The Dockerfile uses multi-stage build with dependency caching. Ensure `mvn dependency:go-offline` layer is cached by not modifying `pom.xml` unnecessarily.

Issue 7: Gmail SMTP fails to send OTP

Cause: Gmail blocks "less secure app access."
Fix: Use a Gmail App Password instead of the regular password. Generate at:
https://myaccount.google.com/apppasswords

Issue 8: PAN card shows encrypted gibberish in database

Expected behavior. PAN cards are AES-256-GCM encrypted at rest. The decrypted value is only available through the API (masked as ABCDE****F).

Issue 9: Frontend build fails on Render

Cause: `VITE_API_BASE_URL` not set during build time.
Fix: Vite env vars must be set at build time (not just runtime). Set them in Render's Environment tab before triggering a deploy.

Issue 10: "Token invalid or expired" after restarting backend

Cause: `JWT_SECRET` changed between restarts.
Fix: Use a consistent `JWT_SECRET` across deployments. Changing it invalidates all existing tokens.

Issue 11: Scheme search returns no results

Cause: Scheme master database not seeded.
Fix: Run `POST /api/schemes/seed` to populate the database from AMFI NAVAll.txt.

Issue 12: Analytics shows "Insufficient data" for all metrics

Cause: No transactions recorded or NAV history not fetched.
Fix: Record at least one transaction. The analytics engine requires transaction data and fetches NAV history on demand from mfapi.in.